
ParBench: Evaluating LLM Parallel Code Translation with Build-Run-Verify Correctness and Augmentation Robustness

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Large language models are increasingly used to port code, but parallel code trans-
2 lation remains difficult to evaluate: repository-level studies conflate translation
3 with build-system generation, while standard code benchmarks do not test GPU
4 programming semantics or benchmark memorization. We present PARBENCH,
5 a kernel-centric benchmark framework for LLM-based translation across CUDA,
6 OpenMP, and OpenCL. ParBench combines declarative benchmark specifications,
7 an end-to-end build-run-verify harness, survey-grounded corpus construction, and
8 AST-driven semantics-preserving source augmentation. The corpus contains 96
9 specifications over 35 kernels from five HPC benchmark suites, yielding six stan-
10 dard translation directions plus two OpenMP-target case-study directions. Across
11 710 primary Qwen 3.5 397B-A17B tasks, ParBench observes a 38.3% pass rate
12 [34.8%, 41.9%], including 64.2% [55.3%, 72.2%] on CUDA-to-OpenMP; GPT-
13 4.1 mini reaches 30.7% [27.1%, 34.6%] across 577 tasks. The dominant bottle-
14 neck is build-stage adaptation (33.9% of all Qwen tasks), while verified semantic
15 failures are smaller but non-negligible (7.2%). Augmentation does not produce a
16 monotonic degradation trend on the balanced CUDA-to-OpenMP subset ($z = 0.0$,
17 $p = 1.0$), and pass@ k sampling rarely rescues hard failures. ParBench therefore
18 exposes both non-trivial current capability and the specific structural failures that
19 remain for reliable parallel code translation.

20 1 Introduction

21 Scientific code increasingly needs to move across parallel programming APIs. CUDA implementa-
22 tions written for NVIDIA GPUs must be ported to CPU OpenMP, OpenCL, HIP, SYCL, OpenMP
23 target offload, or other portability layers as hardware and procurement constraints change. Large lan-
24 guage models (LLMs) are a plausible aid for this work, but parallel translation is not ordinary code
25 translation. Correctness depends on preserving thread-index arithmetic, synchronization, memory-
26 management behavior, reductions, address spaces, launch structure, and host-kernel interfaces.

27 Existing evaluation does not isolate this capability cleanly. Sequential code benchmarks such as Hu-
28 manEval Chen et al. [2021], SWE-bench Jimenez et al. [2024], and TransCoder Rozière et al. [2020]
29 do not evaluate parallel API semantics. Parallel generation benchmarks such as ParEval Nichols
30 et al. [2024] ask models to synthesize code from descriptions, not translate existing parallel struc-
31 ture. Repository-level translation, as in ParEval-Repo Davis et al. [2025], evaluates a realistic work-
32 flow but makes build-system and integration generation the binding constraint: that study reports
33 0% pass@1 on applications larger than 133 SLoC, including XSBench. This leaves a narrower and
34 important question unanswered: can models translate the computational parallel kernel itself when
35 the surrounding executable context is fixed? A fuller related-work comparison is in Appendix D.

Benchmark contamination is a second challenge. HPC suites such as Rodinia Che et al. [2009], XSBench Tramm et al. [2014], and HeCBench Jin and Vetter [2023] are widely replicated, so success on their original source may reflect familiarity with public code rather than robust parallel reasoning. A benchmark for LLM parallel translation should therefore test executable correctness while also probing whether results survive semantics-preserving source variation.

We present PARBENCH, a kernel-centric framework for evaluating LLM-based parallel code translation. ParBench provides fixed build and verification infrastructure, asks the model to translate only the selected kernel files, and judges the output with a build-run-verify harness. A declarative specification describes each kernel/API variant; an AST-based augmentation engine creates transformed but equivalent source variants at five levels; and a unified evaluation pipeline records pass/fail outcomes and diagnostic failure categories. The benchmark corpus is not ad hoc: it is derived from a survey of 35 open-source HPC benchmark repositories and narrowed to suites with equivalent multi-API kernels and automated correctness checks.

This paper makes three contributions.

1. **A kernel-centric parallel translation benchmark.** ParBench provides 96 executable specifications spanning 35 kernels from Rodinia, HeCBench, XSBench, RSBench, and mixbench, with six standard directions across CUDA, OpenMP, and OpenCL plus two OpenMP-target case-study directions. Kernel isolation preserves executable evaluation while avoiding the build-system-generation confound identified by repository-level work.
2. **A spec-driven build-run-verify and augmentation framework.** ParBench uses declarative correctness contracts, a common harness, a failure taxonomy, and six AST-driven semantics-preserving transformations across levels L0–L4 to test robustness beyond surface-form familiarity. Full schema, prompt, and augmentation details appear in Appendix A.
3. **An empirical characterization of current LLM behavior.** On 1,287 primary tasks across Qwen 3.5 397B-A17B and GPT-4.1 mini, ParBench shows non-trivial translation capability under kernel isolation, build-stage adaptation as the dominant bottleneck, strong direction dependence, no monotonic augmentation degradation in the main trend test, and limited rescue from sampling alone.

2 ParBench Overview

ParBench evaluates a translation task of the form: given source files implementing a kernel in one parallel API, produce target files implementing the same computation in another API so that the resulting program builds, runs, and passes the benchmark’s correctness check. Figure 1 summarizes the pipeline.

2.1 Declarative Specifications

Each kernel/API variant is represented by a JSON specification that records identity, provenance, files, build commands, run arguments, and verification strategies. The key design choice is to separate *translation targets* from support files: prompt payload files are shown to the model, target files identify what it must emit, support files and build infrastructure are fixed context, and verification-only files are never shown. This makes correctness definitions explicit and allows the same harness to evaluate every suite. The main verification strategies are exit-code checks and stdout-pattern matching, applied conjunctively; reserved schema fields support numeric comparison, file diffing, and custom scripts. A condensed spec example and schema details are in Appendix A.

2.2 Kernel-Centric Evaluation

ParBench isolates kernel translation from repository reconstruction. The model receives the source kernel files, the target API, target filenames, the target build command, and read-only target infrastructure context needed to match signatures and data structures. Makefiles, shared headers, datasets, and reference outputs are not regenerated by the model. When source and target implementations use different command-line or stdout conventions, the evaluation uses source-side run arguments

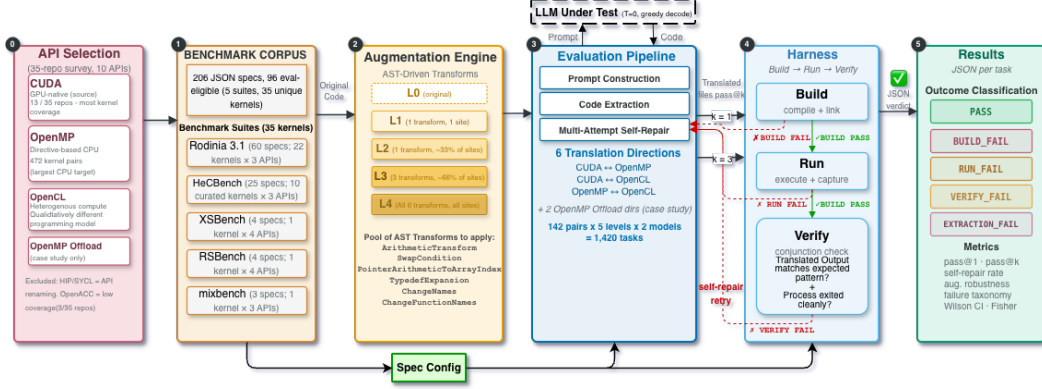


Figure 1: ParBench pipeline. A survey-grounded corpus supplies declarative specs and source files. The augmentation engine creates source variants. The evaluation pipeline prompts an LLM for translated target files, and the harness applies build, run, and verification stages. Failures are classified and, in the primary campaign, returned to the model for bounded self-repair.

and accepts either source- or target-style verification patterns; this avoids penalizing correct translations for superficial interface formatting. The full prompt template and anonymization procedure are given in Appendix G.

2.3 Build, Run, Verify

The harness applies three sequential stages. The build stage invokes the spec-defined build command and checks that the expected executable exists. The run stage executes the binary with the selected input configuration and records stdout, stderr, exit code, and timeout status. The verify stage applies all declared correctness checks. A task passes only if all stages succeed; otherwise it is labeled BUILD_FAIL, RUN_FAIL, VERIFY_FAIL, timeout, or EXTRACTION_FAIL if the LLM response cannot be parsed into target files. This taxonomy distinguishes compilation and API adaptation errors from translations that compile and run but produce incorrect output.

2.4 Augmentation

To probe robustness to surface variation, ParBench augments source code through six libclang-backed transforms: condition swapping, compound-assignment expansion/contraction, local variable and parameter renaming, typedef expansion, pointer-arithmetic/indexing conversion, and static helper-function renaming. L0 is the original source; L1–L4 apply increasing transform and candidate-site density. Re-parsing guards against syntactically invalid rewrites. The augmentation baseline validates semantics preservation: 68 of 88 non-UNKNOWN_FAIL specs across all five suites pass at every augmented level with zero correctness regressions. Transform definitions, level settings, and transform-frequency details appear in Appendix A and Appendix E.3.

3 Benchmark Construction

ParBench’s corpus was assembled through a survey and filtering process rather than by convenience selection. We began with 40 candidate open-source HPC benchmark repositories from proxy-application catalogs, benchmark suites, and conference literature; 35 were analyzed after excluding download failures, duplicates, and insufficiently documented repositories. The survey covered suites, mini-applications, proxy applications, full applications, libraries, and microbenchmarks. Selection required open-source availability, equivalent implementations of the same kernel in multiple APIs, automated build/run/verify support, self-checking output, and domain diversity. Full survey figures, API co-occurrence data, and exclusion rationale are in Appendices B–C.

Table 1: Evaluation corpus. KNOWN_FAIL specs are preserved in metadata but excluded from LLM evaluation. SLoC counts physical non-blank, non-comment lines in the CUDA prompt payload where available.

Suite	Kernels	Specs	PASS	KNOWN_FAIL	SLoC range	APIs
Rodinia	22	60	54	6	195–3,304	CUDA, OMP, OpenCL
HeCBench curated	10	25	23	2	80–235	CUDA, OMP, OMP-target
XSbench	1	4	4	0	1,390	CUDA, OMP, OpenCL, OMP-target
RSBench	1	4	4	0	1,016	CUDA, OMP, OpenCL, OMP-target
mixbench	1	3	3	0	312	CUDA, OMP, OpenCL
Total	35	96	88	8	80–3,304	

The survey also motivates the API focus. CUDA-to-OpenMP provides the largest paradigm-crossing pool in the surveyed material, with 472 kernel-level CUDA–OpenMP pairs across six repositories. CUDA–HIP (633 pairs) and CUDA–SYCL (616 pairs) are larger by raw count, but HIP is close to syntactic renaming and SYCL coverage is concentrated in fewer suites; both are useful portability targets but less diagnostic for the structural question ParBench asks. OpenCL has much lower kernel-level coverage, yet it is retained because its separate host/kernel source model and address-space qualifiers expose a qualitatively different translation challenge. Appendix B gives the full API co-occurrence analysis and exclusion rationale for HIP, SYCL, OpenACC, and OpenMP target offload.

The resulting corpus contains 96 specifications over 35 kernels (Table 1). Rodinia provides the main CUDA/OpenMP/OpenCL substrate with 22 kernels and 60 specs; 54 pass baseline verification and six are KNOWN_FAIL due to toolchain or platform issues such as deprecated CUDA texture APIs, OpenGL dependencies, or pre-existing OpenCL runtime failures. HeCBench contributes 10 curated kernels selected from a much larger multi-API pool for build reliability and domain diversity. XSbench, RSBench, and mixbench add proxy-application and microbenchmark coverage beyond Rodinia.

The corpus spans 80–3,304 SLoC with a median of 271 SLoC, and 24 of 96 specs (25%) require translating more than one source file. Thus kernel-centric evaluation does not reduce the task to toy functions: 31 of 35 kernels exceed the 133-SLoC scale at which ParEval-Repo reports repository-level failure Davis et al. [2025]. Multi-file coordination remains an independent challenge and is analyzed in Appendix E.5.

4 Experimental Setup

We evaluate two provider-diverse models: Qwen 3.5 397B-A17B through Together AI and GPT-4.1 mini through Azure OpenAI. The primary campaigns use temperature 0, no chain-of-thought mode, and up to three self-repair attempts after the initial translation. A separate pass@ k sweep for Qwen 3.5 uses temperature 0.7, L0 only, three independent samples, and no self-repair. Model and hardware details are in Appendix Tables 10 and 11; token and cost accounting is in Appendix I.

The standard evaluation matrix covers six directions: CUDA↔OpenMP, CUDA↔OpenCL, and OpenMP↔OpenCL. Two OpenMP-target directions are included as case studies because they are available only in a smaller subset and require a different compiler stack. For Qwen 3.5, the primary campaign contains 142 L0 source-target pairs across all directions and five augmentation levels, for 710 tasks. GPT-4.1 mini covers 577 tasks across seven directions and five augmentation levels.

The primary metric is greedy pass rate: the fraction of tasks that pass build, run, and verify after deterministic decoding and bounded self-repair. We report Wilson 95% confidence intervals for proportions. Direction asymmetry is tested with McNemar’s exact test on paired kernels; augmentation trend is tested with Cochran-Armitage on the balanced CUDA-to-OpenMP subset; and cross-model aggregate differences are tested with chi-squared tests. Performance speedup is intentionally out of scope: wall-clock measurements in the current harness conflate kernel runtime with I/O, allocation, launch overhead, and scheduling noise.

Table 2: Overall pass rates across primary campaigns. Qwen covers 710 tasks; GPT-4.1 mini covers 577 tasks. KNOWN_FAIL specs are excluded.

Model	PASS	BUILD_FAIL	RUN_FAIL	VERIFY_FAIL	EXTR_FAIL	ERROR	Total	Rate	95% Wilson CI
Qwen 3.5 397B-A17B	272	241	144	51	1	1	710	38.3%	[34.8%, 41.9%]
GPT-4.1 mini	177	267	54	79	0	0	577	30.7%	[27.1%, 34.6%]
Aggregate	449	508	198	130	1	1	1,287	34.9%	[32.3%, 37.5%]

Table 3: Pass rates by direction across all augmentation levels. The OpenMP-target rows are case studies rather than part of the standard six-direction matrix.

Direction	Qwen 3.5	GPT-4.1 mini	L0 pairs	Note
CUDA→OMP	64.2% (77/120) [55.3%, 72.2%]	56.5% (48/85) [45.9%, 66.5%]	24	Primary
OMP→CUDA	52.5% (63/120) [43.6%, 61.2%]	14.9% (16/107) [9.4%, 22.9%]	24	
OpenCL→OMP	38.9% (35/90) [29.5%, 49.2%]	46.2% (37/80) [35.8%, 57.1%]	18	
OMP→OpenCL	27.8% (25/90) [19.6%, 37.8%]	37.5% (30/80) [27.7%, 48.4%]	18	
CUDA→OpenCL	20.0% (20/100) [13.3%, 28.9%]	33.3% (30/90) [24.4%, 43.6%]	20	
OpenCL→CUDA	6.0% (6/100) [2.8%, 12.5%]	1.2% (1/85) [0.2%, 6.4%]	20	
CUDA→OMP-target	17.5% (7/40) [8.8%, 31.9%]	—	8	Case study
OMP-target→CUDA	78.0% (39/50) [64.8%, 87.2%]	30.0% (15/50) [19.1%, 43.8%]	10	Case study

5 Results

5.1 Overall Performance

Kernel-centric evaluation reveals capability that repository-level evaluation hides. Qwen 3.5 passes 272 of 710 tasks, or 38.3% [34.8%, 41.9%], across five suites, all augmentation levels, and eight directions including case studies (Table 2). GPT-4.1 mini passes 177 of 577 tasks, or 30.7% [27.1%, 34.6%]. The aggregate cross-model difference is statistically significant ($\chi^2 = 7.83$, $p = 0.005$), but the effect size is small (Cohen’s $h = 0.16$), so the more informative comparison is by direction.

The comparison to repository-level work is the main methodological point. ParEval-Repo reports 0% pass@1 for all models on applications larger than 133 SLoC Davis et al. [2025]. ParBench evaluates many kernels above that size but fixes the build and integration scaffold, yielding 38.3% overall and 64.2% [55.3%, 72.2%] on Qwen CUDA-to-OpenMP. This does not make the task easy; it separates kernel translation failures from full-repository reconstruction failures.

5.2 Direction Dependence

Performance is strongly direction-dependent (Table 3). For Qwen 3.5, CUDA-to-OpenMP is the strongest standard direction at 64.2%, followed by OpenMP-to-CUDA at 52.5%. OpenCL directions are harder, especially OpenCL-to-CUDA at 6.0%. This pattern matches the structural asymmetry of the APIs: translating from CUDA to OpenMP often removes explicit device memory management and thread launch syntax, whereas translating to CUDA requires introducing those details coherently. OpenCL adds a different source structure and address-space model, and OpenCL-target translations often surface kernel compilation errors only at runtime through `clBuildProgram()`.

At the kernel level, difficulty is heterogeneous. Table 4 shows the five easiest and five hardest kernels under Qwen 3.5. The easier kernels tend to be regular stencil, graph, or solver kernels with structured loop nests; the hard set includes irregular control flow, domain-specific data structures, brittle synchronization, and multi-file dependencies. The full 31-kernel table is in Appendix Table 17.

5.3 File Coordination Effects

Kernel isolation removes repository build generation from the task, but it does not remove cross-file reasoning. Some ParBench translations still require coordinated edits across host wrappers, kernel files, and helper interfaces. Table 5 shows that this dimension strongly affects Qwen 3.5 outcomes: single-file translations pass at 51.3%, while multi-file classes range from 36.3% down to 8.0%. A chi-squared test over the four file-cardinality classes yields $\chi^2 = 82.73$ ($df = 3$, $p = 0.005$). Thus, multi-file coordination is a separate bottleneck from build-system generation: even when the build

Table 4: Representative per-kernel pass rates for Qwen 3.5 across all directions and augmentation levels.

Suite	Kernel	Total	PASS	Fail	Rate
<i>Easiest kernels</i>					
HeCBench	stencil1d	15	15	0	100.0%
HeCBench	floydwarshall	20	16	4	80.0%
HeCBench	heat2d	20	15	5	75.0%
HeCBench	iso2dfd	20	15	5	75.0%
Rodinia	hotspot	30	21	9	70.0%
<i>Hardest kernels</i>					
Rodinia	gaussian	10	0	10	0.0%
Rodinia	heartwall	30	0	30	0.0%
Rodinia	myocyte	30	0	30	0.0%
RSBench	rsbench	30	0	30	0.0%
XSbench	xsbench	30	0	30	0.0%

Table 5: Qwen 3.5 pass rate by source/target file cardinality.

Class	Tasks	Pass rate	Typical challenge
single_file	380	51.3%	within-file API mapping
multi_to_single	135	36.3%	consolidating split source structure
single_to_multi	135	13.3%	emitting coherent split targets
multi_to_multi	50	8.0%	cross-file signature consistency

183 scaffold is fixed, the model must keep declarations, launch sites, and function signatures consistent
 184 across emitted files.

185 The pattern clarifies why XSbench and RSBench remain difficult despite kernel-level evaluation.
 186 They are not blocked by repository Makefiles in ParBench, but their multi-file structure and domain-
 187 specific data abstractions still require coordinated program synthesis. Appendix E.5 gives a detailed
 188 multi-file failure case study.

189 5.4 Failure Modes and Self-Repair

190 Build-stage adaptation is the dominant bottleneck. In the Qwen campaign, BUILD_FAIL accounts
 191 for 241/710 tasks (33.9%) and 55.0% of all failures. The largest build-failure subcategories are
 192 undeclared identifiers (56 tasks), missing headers (55), and linker errors (49). These often indicate
 193 incomplete API-surface adaptation: retained CUDA identifiers in OpenMP translations, missing
 194 OpenCL address-space qualifiers, or mismatched target-side dependencies.

195 VERIFY_FAIL is smaller but important: 51/710 tasks (7.2%) compile and run but fail correctness.
 196 Of these, 46 produce wrong numerical output and 5 produce missing output. These cases are the
 197 strongest evidence that compilation alone is insufficient: exit-code-only or compile-only checks
 198 would misclassify them as successful translations. Detailed examples include OpenCL vector type
 199 conflicts in cfd, thread mapping errors in hotspot, and output-structure mistakes in gaussian;
 200 these case studies appear in Appendix E.

201 Self-repair improves results but does not solve the task. For Qwen 3.5, the first-attempt pass rate is
 202 22.5% [19.6%, 25.8%]; after up to three diagnostic retries, it rises to 38.3% [34.8%, 41.9%], a 15.8
 203 percentage-point gain and 70.0% relative increase. Of 550 initially failing tasks, 112 are repaired,
 204 while seven regressions occur. Compiler feedback is most useful for build errors, whereas runtime
 205 and verification failures offer weaker diagnostic signals. Transition matrices and direction-level
 206 repair rates are in Appendix Tables 12–13.

207 5.5 Augmentation Robustness and Sampling

208 Augmentation does not show the monotonic degradation expected if baseline success were mainly
 209 driven by exact surface familiarity. On the balanced CUDA-to-OpenMP subset ($n = 24$ kernels
 210 per level), Qwen pass counts are 16, 14, 17, 14, and 16 at L0–L4. The Cochran-Armitage trend

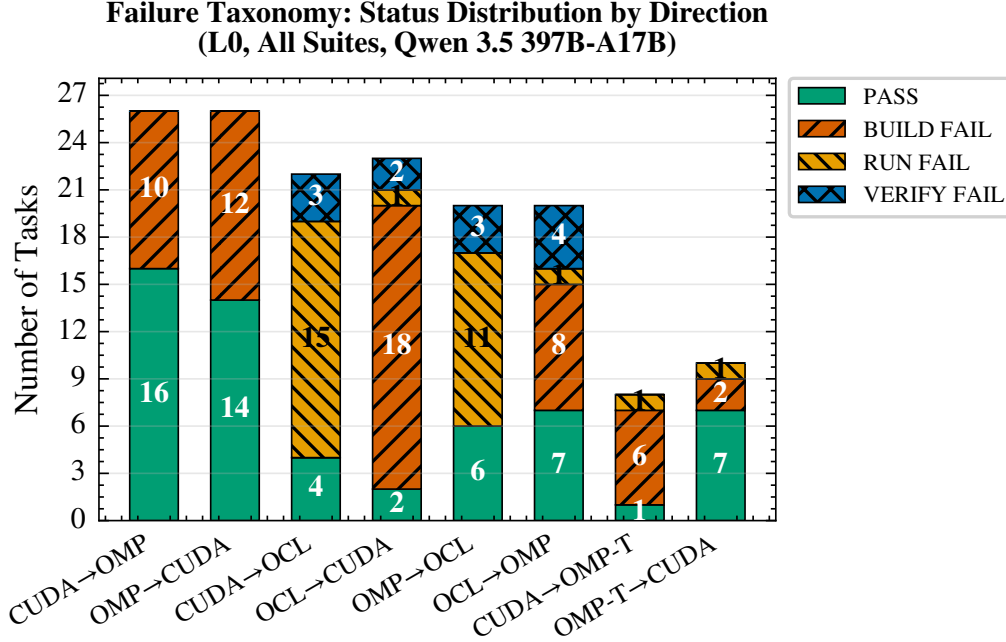


Figure 2: Qwen 3.5 outcome taxonomy across 710 primary tasks. Build-stage failures are the largest non-pass category. GPT-4.1 mini failure visualizations are in Appendix H.

test yields $z = 0.0$, $p = 1.0$. This is a null result, not proof of invariance: with $n = 24$ per level, the test has 80% power only for monotonic trends above roughly 34 percentage points. Still, the observed stability argues against a simple memorized-source explanation. Full augmentation rates and heatmaps appear in Appendix Table 14 and Figure 17.

Sampling also has limited effect. In the Qwen pass@ k sweep (142 L0 direction-kernel pairs, three samples each at temperature 0.7, no self-repair), 103 pairs (72.5%) are hard failures with no passing sample, 22 (15.5%) are partial successes, and 17 (12.0%) pass in all samples. No pair has pass@1 equal to zero but pass@3 above zero. Additional samples therefore do not dramatically rescue the task; many failures appear structural at the kernel-direction level. Full pass@ k results are in Appendix Table 15.

6 Related Work

ParBench sits at the intersection of code translation benchmarks, LLM-for-HPC evaluation, and robustness testing. General-purpose code benchmarks have established execution-based evaluation as the standard for measuring functional correctness, but their task distributions are far from parallel HPC translation. HumanEval Chen et al. [2021] and TransCoder Rozière et al. [2020] operate at function granularity, while SWE-bench Jimenez et al. [2024] evaluates repository-level issue resolution in general software projects. These benchmarks are valuable, but they do not exercise GPU memory models, thread synchronization, API-specific launch structure, or host-device coordination.

Parallel-code benchmarks are closer but still answer different questions. ParEval Nichols et al. [2024] studies synthesis of parallel code from natural language tasks, which tests whether a model can produce a solution but not whether it can preserve an existing implementation’s decomposition across APIs. ParEval-Repo Davis et al. [2025] studies full-repository translation and demonstrates a severe integration bottleneck: models fail once they must recreate build systems and repository scaffolding. ParBench is complementary to both. It preserves executable evaluation but fixes build infrastructure so the measured object is kernel translation.

LASSI Dearing et al. [2024] is the nearest prior work in granularity and verification. It evaluates an agentic self-correction pipeline for CUDA/OpenMP-target translation on 10 HeCBench kernels, whereas ParBench evaluates a broader survey-grounded corpus, six standard directions, bounded self-repair, and augmentation robustness. CodeRosetta TehraniJamsaz et al. [2024] and HPC-Coder-V2 Chaturvedi et al. [2024] demonstrate specialized modeling and fine-tuning for parallel code, but ParBench provides an execution-based substrate for comparing such models against general-purpose LLMs. Extended comparisons are in Appendix D.

Finally, ParBench follows recent concerns about benchmark validity. Public code benchmarks can be memorized, and surface-form perturbations can reveal when evaluation depends on brittle pattern matching. ParBench’s augmentation engine is not a complete solution to training-data contamination because it cannot rule out algorithm-level familiarity, but it provides a concrete robustness probe for widely circulated HPC kernels. Its correctness-only scope also differs from efficiency-oriented work such as TRACY Gong et al. [2025], which shows that functional correctness and performance are separable.

7 Discussion, Limitations, and Conclusion

ParBench establishes that LLMs can perform non-trivial parallel code translation when evaluation isolates the kernel and preserves executable correctness. It also shows where current systems fail. The main obstacle is not only semantic reasoning over parallel computation, nor only build syntax, but a spectrum of incomplete API adaptation, host-kernel coordination, and genuine correctness errors. Direction-level results matter: aggregate pass rates conceal that CUDA-to-OpenMP is much more tractable than OpenCL-to-CUDA, and that model comparisons differ by direction rather than uniformly across the benchmark.

The benchmark is correctness-focused. It does not measure speedups, occupancy, memory bandwidth, or efficiency; correct translated code may still be unusably slow, a concern also raised by TRACY Gong et al. [2025]. The corpus is also finite. Some directions have small paired samples, the augmentation trend test is underpowered for moderate effects, and several specs are marked KNOWN_FAIL because of platform or toolchain constraints. Rodinia and related suites are likely present in training data, and augmentation tests surface-form robustness rather than deeper algorithmic memorization. Finally, the experiments cover two models and one bounded self-repair protocol; broader model families, agentic systems such as LASSI Dearing et al. [2024], fine-tuned HPC models Chaturvedi et al. [2024], and efficiency-aware pipelines remain important follow-up evaluations.

ParBench is intended as evaluation infrastructure rather than a new translation method. Its contribution is to make the measurement problem sharper: kernel-centric isolation avoids repository-level build confounds, declarative specs make correctness reproducible, augmentation probes robustness, and the harness exposes where translations fail. The initial results show that current LLMs have real but uneven parallel translation capability, with build adaptation, direction asymmetry, and multi-file coordination as central barriers to reliable use.

References

- Aman Chaturvedi, Daniel Nichols, Siddharth Singh, and Abhinav Bhatele. HPC-Coder-V2: Studying code LLMs across low-resource parallel languages. arXiv preprint arXiv:2412.15178, 2024. URL <https://arxiv.org/abs/2412.15178>.
- Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. Rodinia: A benchmark suite for heterogeneous computing. In *2009 IEEE International Symposium on Workload Characterization (IISWC)*, pages 44–54. IEEE, 2009. doi: 10.1109/IISWC.2009.5306797. URL <https://doi.org/10.1109/IISWC.2009.5306797>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex

287 Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders,
288 Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa,
289 Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder,
290 Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021. URL <https://arxiv.org/abs/2107.03374>.

293 Joshua H. Davis, Daniel Nichols, Ishan Khillan, and Abhinav Bhatele. On the limits of LLM-based
294 repository-level HPC code translation. In *Proceedings of the 54th International Conference on*
295 *Parallel Processing (ICPP)*, pages 94–103. ACM, 2025. doi: 10.1145/3754598.3754669. URL
296 <https://doi.org/10.1145/3754598.3754669>.

297 Matthew T. Dearing, Yiheng Tao, Xingfu Wu, Zhiling Lan, and Valerie Taylor. LASSI: An LLM-
298 based automated self-correcting pipeline for translating parallel scientific codes. In *2024 IEEE*
299 *International Conference on Cluster Computing Workshops (CLUSTER Workshops)*, pages 136–
300 143. IEEE, 2024. doi: 10.1109/CLUSTERWorkshops61563.2024.00029. URL <https://doi.org/10.1109/CLUSTERWorkshops61563.2024.00029>.

302 Zhihao Gong, Zeyu Sun, Dong Huang, Qingyuan Liang, Jie M. Zhang, and Dan Hao.
303 TRACY: Benchmarking execution efficiency of LLM-based code translation. arXiv preprint
304 arXiv:2508.11468, 2025. URL <https://arxiv.org/abs/2508.11468>.

305 Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik
306 Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceed-*
307 *ings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024. doi:
308 10.48550/arXiv.2310.06770. URL <https://openreview.net/forum?id=VTF8yNQM66>.

309 Zheming Jin and Jeffrey S. Vetter. A benchmark suite for improving performance portability of the
310 SYCL programming model. In *2023 IEEE International Symposium on Performance Analysis of*
311 *Systems and Software (ISPASS)*, pages 325–327. IEEE, 2023. doi: 10.1109/ISPASS57527.2023.
312 00041. URL <https://doi.org/10.1109/ISPASS57527.2023.00041>.

313 Tal Kadosh, Nadav Schneider, Niranjana Hasabnis, Timothy Mattson, Yuval Pinter, and Gal Oren.
314 Advising OpenMP parallelization via a graph-based approach with transformers. In *OpenMP:*
315 *Advanced Task-Based, Device and Compiler Programming (IWOMP 2023)*, volume 14114 of *Lec-*
316 *ture Notes in Computer Science*, pages 3–17. Springer, 2023. doi: 10.1007/978-3-031-40744-4_1.
317 URL https://doi.org/10.1007/978-3-031-40744-4_1.

318 Daniel Nichols, Joshua H. Davis, Zhaojun Xie, Arjun Rajaram, and Abhinav Bhatele. Can large
319 language models write parallel code? In *Proceedings of the 33rd International Symposium on*
320 *High-Performance Parallel and Distributed Computing (HPDC)*, pages 281–294. ACM, 2024.
321 doi: 10.1145/3625549.3658689. URL <https://doi.org/10.1145/3625549.3658689>.

322 Baptiste Rozière, Marie-Anne Lachaux, Loïc Chaneussot, and Guillaume Lample. Un-
323 supervised translation of programming languages. In *Advances in Neural Informa-*
324 *tion Processing Systems 33 (NeurIPS)*, 2020. doi: [https://dl.acm.org/doi/10.5555/](https://dl.acm.org/doi/10.5555/3495724.3497454)
325 3495724.3497454. URL [https://proceedings.neurips.cc/paper/2020/hash/](https://proceedings.neurips.cc/paper/2020/hash/ed23fbf18c2cd35f8c7f8de44f85c08d-Abstract.html)
326 ed23fbf18c2cd35f8c7f8de44f85c08d-Abstract.html.

327 Ali TehraniJamsaz, Arijit Bhattacharjee, Le Chen, Nesreen K. Ahmed, Amir Yazdanbakhsh, and Ali
328 Jannesari. CodeRosetta: Pushing the boundaries of unsupervised code translation for parallel pro-
329 gramming. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. doi: <https://doi.org/10.52202/079017-3202>. URL <https://openreview.net/forum?id=V6hrg409gg>.

331 John R. Tramm, Andrew R. Siegel, Tanzima Islam, and Martin Schulz. XSBench – the development
332 and verification of a performance abstraction for Monte Carlo reactor analysis. In *Proceedings*
333 *of the International Conference on the Physics of Reactors (PHYSOR)*, Kyoto, Japan, 2014. doi:
334 <https://www.mcs.anl.gov/papers/P5064-0114.pdf>.

```

{
  "identity": {
    "unique_id": "rodinia-bfs-cuda",
    "parallel_api": "cuda",
    "source_suite": "rodinia"
  },
  "files": {
    "prompt_payload": ["bfs.cu", "kernel.cu", "kernel2.cu"],
    "support_files": ["Makefile"],
    "translation_targets": ["bfs.cu", "kernel.cu", "kernel2.cu"]
  },
  "build": {
    "commands": {"build": "make CUDA_DIR=..."},
    "outputs": {"executable": "./bfs.out"}
  },
  "run": {
    "executable": "./bfs.out",
    "timeout_seconds": 300,
    "input_configurations": {
      "correctness": {
        "arguments": ["../data/bfs/graph1MW_6.txt"]
      }
    }
  },
  "verification": {
    "strategies": [
      {"type": "exit_code", "expected": 0}
    ]
  }
}

```

Listing 1: Condensed specification structure for rodinia-bfs-cuda.

A Framework and Evaluation Details

This appendix expands the compact framework description in Section 2. It records the spec structure, augmentation level definitions, and implementation details that are useful for reproducing the benchmark but too detailed for the main narrative.

A.1 Specification Schema

Each benchmark/API variant is encoded by a JSON specification. The schema defines five field groups:

- **Identity and provenance:** globally unique identifier, source suite, kernel name, target API, repository URL, and commit.
- **File partitioning:** prompt payload files, translation targets, support files, and verification-only files.
- **Build:** build system, clean/configure/build commands, environment variables, and expected executable path.
- **Run:** executable, input configuration, command-line arguments, timeouts, and optional environment variables.
- **Verification:** ordered correctness strategies, currently exit-code and stdout-pattern checks applied conjunctively.

A.2 Augmentation Levels

Table 6 gives the exact L0–L4 augmentation definitions. L1 selects one transform and one eligible candidate site. L2–L4 select increasing fractions of transforms and candidate sites, using a minimum of one site when a selected transform has at least one eligible candidate.

Table 6: Augmentation level definitions.

Level	Transform selection	Candidate fraction	Description
L0	None	0%	Unmodified source code baseline
L1	1 transform	1 site	Minimal source perturbation
L2	33% of transforms	33% of candidates	Moderate perturbation
L3	66% of transforms	66% of candidates	Heavy perturbation
L4	All transforms	100% of candidates	Maximum configured perturbation

A.3 Prompt Anonymization

Before prompt construction, ParBench strips source comments, removes the kernel name and description, replaces source and target filenames with generic labels, genericizes support-file names, and anonymizes kernel identifiers in build commands. These measures complement source augmentation: anonymization reduces benchmark identity leakage, while augmentation changes the source surface form. The complete prompt template is reproduced in Appendix G.

B API Selection Rationale

This appendix provides the complete quantitative evidence underlying ParBench’s selection of CUDA, OpenMP, and OpenCL as target parallel programming APIs. Section 3 in the main paper summarizes the corpus selection process; here we present the expanded survey data and kernel-level analysis that informed the final selection.

B.1 Full API Co-Occurrence Data

The benchmark landscape survey progressed through multiple iterations, expanding from an initial seed of 21 repositories to a comprehensive survey of over 80 benchmark suites, mini-applications, proxy applications, libraries, and full applications. This broader survey captured 30+ distinct parallel programming APIs, providing a panoramic view of the HPC benchmarking ecosystem.

Figure 3 shows the full expanded survey. The dominant pattern is a dense co-occurrence cluster centered on MPI, OpenMP, CUDA, and HIP, with sparser connectivity among directive-based (OpenACC, OpenMP Target), portability (Kokkos, RAJA, SYCL), and niche APIs.

The co-occurrence structure reveals three natural groupings. **Core cluster:** MPI, OpenMP, CUDA, HIP, OpenACC, and OpenCL form a tightly interconnected group, co-occurring frequently across the benchmark suites with the richest multi-API material (HeCBench, Rodinia, RAJAPerf, BabelStream). **Portability periphery:** SYCL, Kokkos, RAJA, and OpenMP Target connect to the core cluster with weaker co-occurrence, concentrated in a smaller number of suites (primarily RAJAPerf, BabelStream, and HeCBench). **Isolated APIs:** OCCA, Thrust, Chapel, Coarray Fortran, Charm++, C++ AMP, UPC, OpenSHMEM, HMPP, Python, HLS, GSParLib, STM/TM, and Serial have co-occurrence below 3 with all other APIs.

Figure 4 presents the per-API coverage in a ranked bar chart, making the quantitative dominance of the top-3 APIs clear.

Kernel-Level Co-Occurrence

The repository-level analysis in Figures 3–4 counts how many *repositories* provide implementations in a given API pair, but does not capture the depth of coverage within each repository. A suite like HeCBench provides 516 individual kernels, each potentially implemented in multiple APIs, while a single-kernel mini-application like miniBUDE provides only 1 kernel across 12+ APIs. For translation evaluation, the kernel count is the relevant unit: it determines how many independent evaluation instances are available per direction.

Figure 5 provides the strongest quantitative evidence for API selection. The kernel-level data reveals a clear hierarchy:

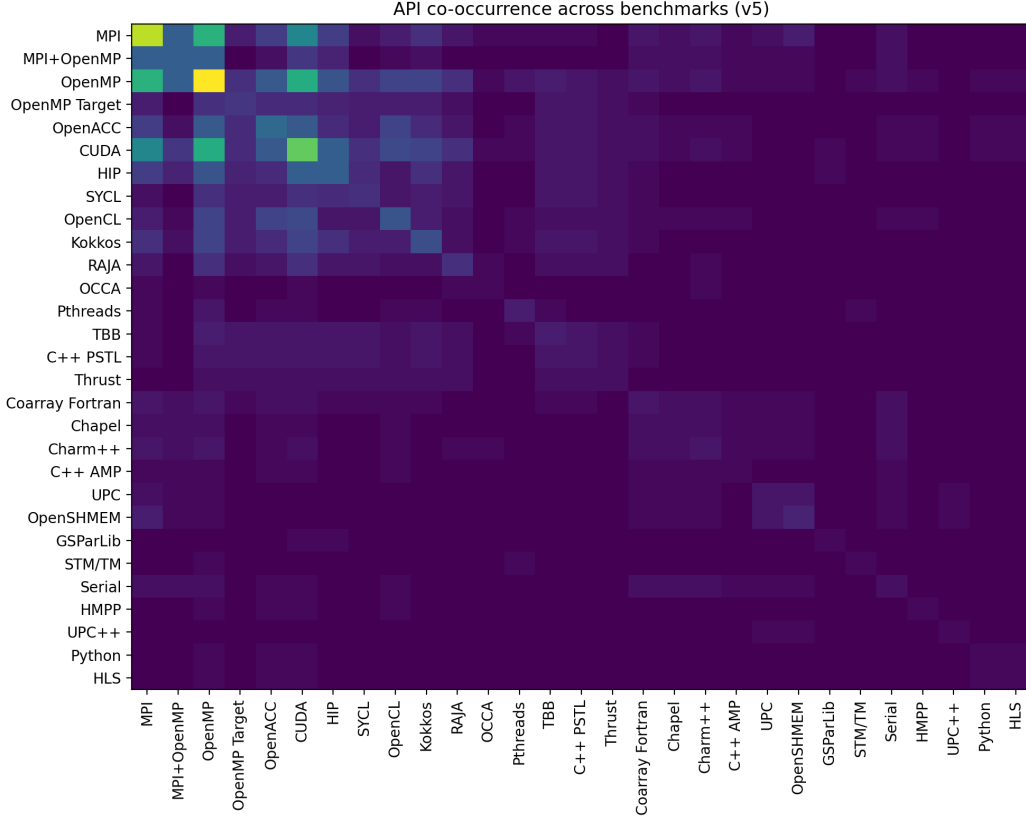


Figure 3: Expanded API co-occurrence heatmap across all surveyed repositories. Cell values indicate the number of repositories providing implementations in both the row and column APIs. The concentrated cluster of high co-occurrence among MPI, OpenMP, CUDA, HIP, and OpenACC in the upper-left quadrant reveals the core API ecosystem that dominates multi-API benchmark suites. Peripheral APIs such as Chapel, STM/TM, HLS, and Python show near-zero co-occurrence, reflecting their niche status in HPC benchmarking.

- **Tier 1 (>600 kernel pairs):** CUDA–HIP (633), CUDA–SYCL (616), HIP–SYCL (615). These pairs are dominated by HeCBench’s systematic multi-API coverage.
- **Tier 2 (~450 kernel pairs):** CUDA–OpenMP (472), HIP–OpenMP (453), SYCL–OpenMP (453). OpenMP’s CPU threading model provides the deepest pool for *paradigm-crossing* translation evaluation.
- **Tier 3 (~100 kernel pairs):** OpenMP Target and Sequential at 106 each, concentrated in HeCBench.
- **Tier 4 (<30 kernel pairs):** OpenCL at 27 (CUDA–OpenCL) and 24 (OpenMP–OpenCL), with an OpenCL diagonal of 27 total kernels.

The network visualization in Figure 6 makes this tiered structure visually apparent.

B.2 Translation Difficulty Taxonomy

The choice of API pairs for evaluation is motivated not only by data availability (Section B.1) but by the *structural transformation complexity* each translation direction demands. We classify translation directions into four categories:

Syntactic renaming (e.g., CUDA to HIP): Near-1:1 API call mapping. `cudaMalloc` becomes `hipMalloc`; kernel launch syntax is preserved verbatim; thread-index arithmetic (`threadIdx.x`,

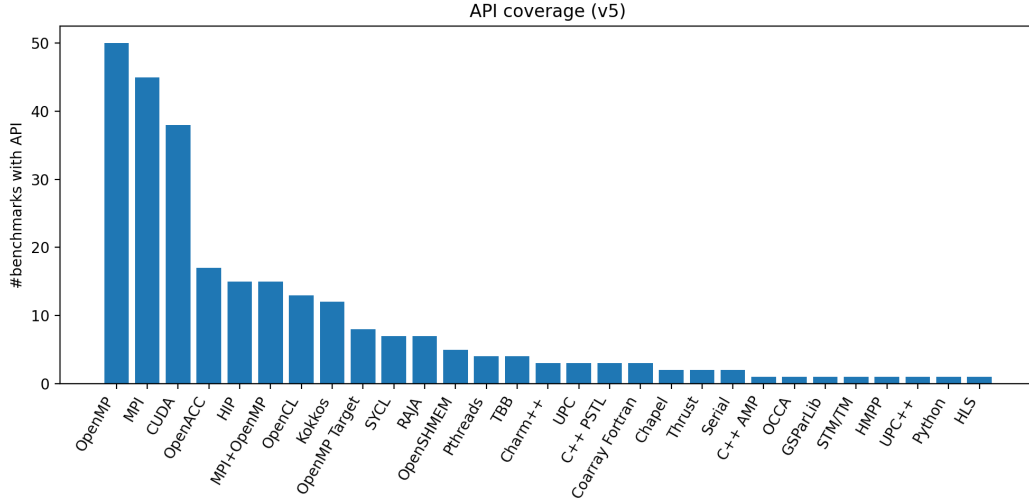


Figure 4: API coverage across the expanded benchmark survey. OpenMP (~50 benchmarks), MPI (~45), and CUDA (~38) dominate, with OpenACC (~17), HIP (~15), and MPI+OpenMP (~15) forming a second tier. OpenMP Target (~8), SYCL (~7), Kokkos (~7), and RAJA (~7) form a third tier. APIs below 5 benchmarks were excluded from evaluation consideration due to insufficient benchmark material for statistical evaluation.

blockIdx.x) maps directly to HIP equivalents. This category tests API name lookup, not parallel reasoning.

```

412 // CUDA                                // HIP (syntactic rename)
413 cudaMalloc(&d_a, size);                hipMalloc(&d_a, size);
414 kernel<<<grid, block>>>(d_a);          hipLaunchKernelGGL(kernel,
415                                     grid, block, 0, 0, d_a);
416 cudaMemcpy(h_a, d_a, ...);             hipMemcpy(h_a, d_a, ...);
417

```

Listing 2: CUDA to HIP: syntactic renaming.

Paradigm translation (e.g., CUDA to OpenMP): SPMD to fork-join model transformation. Explicit GPU thread indexing must be converted to loop-based parallelism with OpenMP directives. Shared memory and synchronization primitives (`__shared__`, `__syncthreads()`) must be replaced with stack-allocated arrays and implicit barrier semantics. Memory management (`cudaMalloc/cudaMemcpy/cudaFree`) is entirely eliminated.

```

424 // CUDA                                // OpenMP
425 __global__ void kern(float* a, int n) { void kern(float* a, int n) {
426     int i = blockIdx.x * blockDim.x      #pragma omp parallel for
427     + threadIdx.x;                       for (int i = 0; i < n; i++) {
428     if (i < n) a[i] = a[i] * 2.0f;        a[i] = a[i] * 2.0f;
429 }                                         }
430
431

```

Listing 3: CUDA to OpenMP: paradigm translation.

Architecture split (e.g., CUDA to OpenCL): Unified single-source CUDA must be split into separate host code (OpenCL runtime API calls for platform/device/context/queue management) and kernel source (.cl files with OpenCL C syntax). The memory model changes from CUDA's unified virtual addressing to OpenCL's explicit buffer objects. Kernel launch syntax transforms from `<<grid, block>>` to `clEnqueueNDRangeKernel` with explicit work-group sizing.

Directive insertion (e.g., sequential C to OpenMP): Adding parallelism to sequential code by identifying parallelizable loops and inserting appropriate `#pragma omp` directives with correct data-

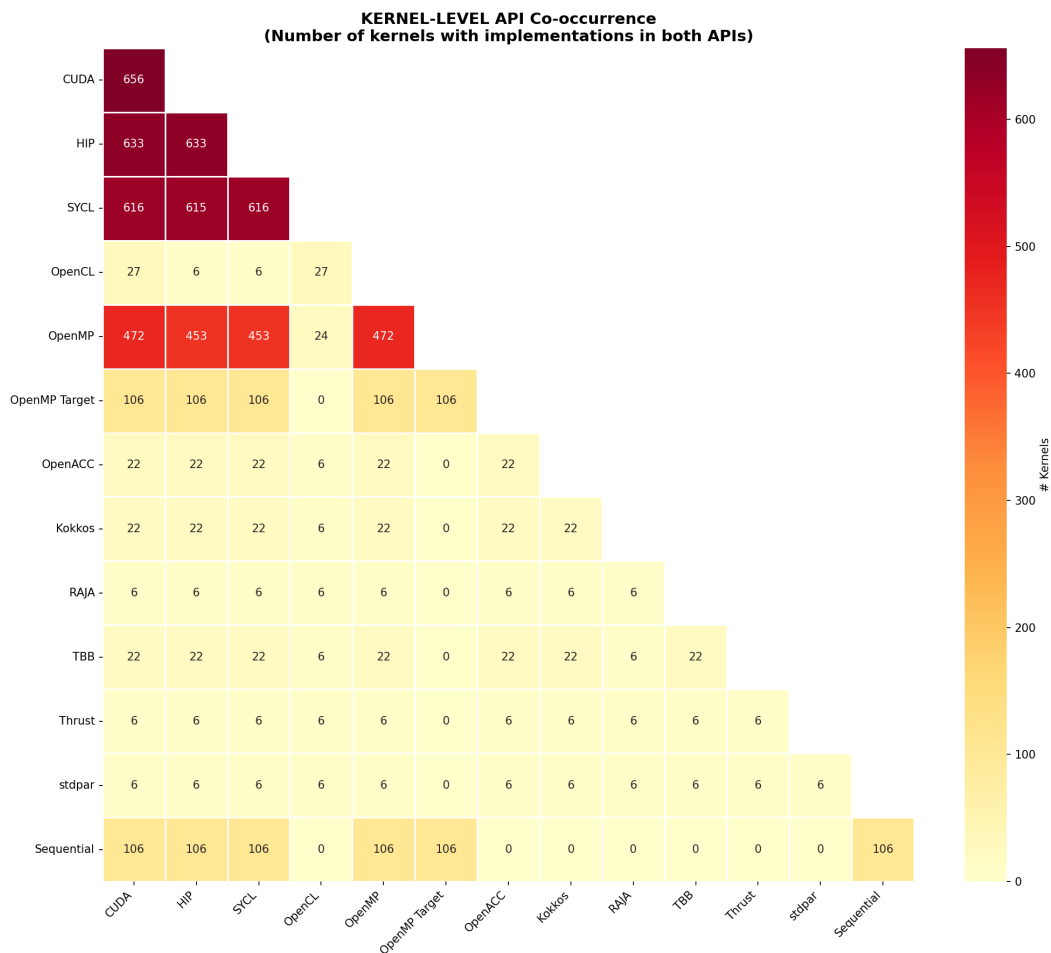


Figure 5: Kernel-level API co-occurrence matrix. Each cell counts the number of individual kernels with verified equivalent implementations in both the row and column APIs. The CUDA–HIP pair leads with 633 shared kernels, followed by CUDA–SYCL (616), CUDA–OpenMP (472), and HIP–SYCL (615). OpenCL has dramatically lower kernel-level coverage: only 27 kernels share implementations with CUDA, and 24 with OpenMP. OpenMP Target provides 106 shared kernels with each of CUDA, HIP, SYCL, and OpenMP, concentrated entirely in HeCBench. Sequential reference implementations exist for 106 kernels.

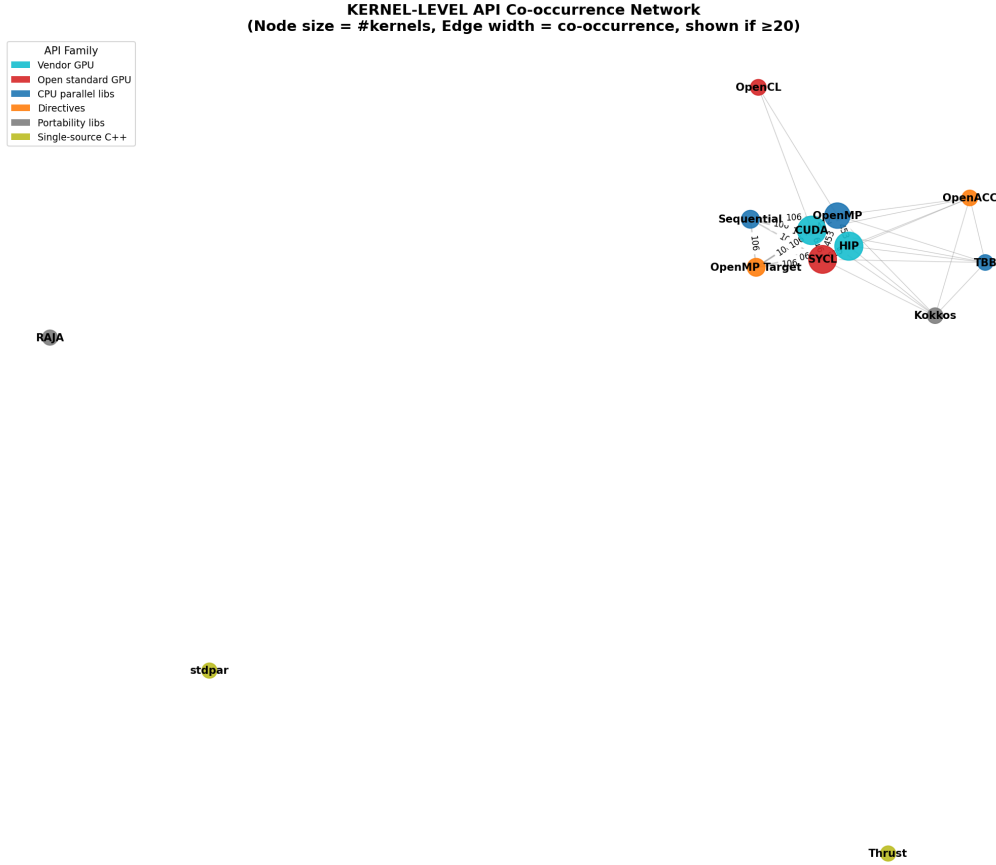


Figure 6: Kernel-level API co-occurrence network. Edges shown for ≥ 20 kernel pairs; edge width proportional to count. The tight CUDA/HIP/SYCL/OpenMP cluster reflects HeCBench and RAJA-Perf’s uniform multi-API coverage. OpenCL appears as a peripheral node with weak connections (27 CUDA–OpenCL kernel pairs, just above the 20-kernel threshold). RAJA, stdpar, and Thrust appear as isolated nodes with fewer than 20 kernel-level pairs.

440 sharing clauses (`shared`, `private`, `reduction`). This is the inverse of the paradigm translation
441 direction and tests whether the LLM can *discover* parallelism rather than *translate* it.

442 ParBench focuses on paradigm translation (CUDA/OpenMP/OpenCL cross-translations) because
443 this category maximizes the semantic challenge while remaining tractable for automated verifica-
444 tion. Syntactic renaming (CUDA to HIP) is too easy to be informative; architecture split (CUDA
445 to OpenCL) combines paradigm translation with source-file restructuring, adding confounding com-
446 plexity.

447 B.3 HIP and SYCL Exclusion Rationale

448 Despite the large kernel-level co-occurrence counts for CUDA–HIP (633) and CUDA–SYCL (616),
449 these pairs were excluded from the primary evaluation for distinct reasons.

450 **HIP exclusion:** CUDA-to-HIP translation is predominantly a syntactic renaming task. The AMD
451 HIP API was explicitly designed as a drop-in replacement for CUDA, preserving the SPMD pro-
452 gramming model, kernel launch syntax, and memory management API. Automated tools such as
453 `hipify-perl` and `hipify-clang` achieve near-perfect conversion rates. Evaluating LLMs on
454 CUDA-to-HIP translation would measure API name lookup ability rather than parallel program-
455 ming reasoning—the core capability ParBench is designed to assess.

456 **SYCL exclusion:** While SYCL introduces a genuinely different programming model (single-source
 457 C++ with lambda-based kernel dispatch and buffer/accessor memory management), it preserves the
 458 GPU execution model. CUDA-to-SYCL translation is more complex than CUDA-to-HIP but re-
 459 mains within the same architectural paradigm (GPU offload). Furthermore, SYCL benchmark cover-
 460 age is concentrated almost entirely in HeCBench, limiting the diversity of evaluation instances. The
 461 CUDA-to-OpenMP direction provides a more fundamental paradigm crossing (GPU SPMD to CPU
 462 fork-join) with better benchmark diversity across Rodinia, HeCBench, XSBench, and RSBench.

463 B.4 OpenACC Exclusion

464 OpenACC was excluded despite appearing in approximately 17 benchmarks in the expanded survey
 465 (Figure 4). Three factors motivated this decision:

- 466 1. **Limited co-occurrence with CUDA/OpenMP:** Of the 17 benchmarks with OpenACC,
 467 only 3 also provide both CUDA and OpenMP implementations. This yields insufficient
 468 material for statistically meaningful cross-API evaluation.
- 469 2. **Paradigm overlap with OpenMP:** OpenACC’s directive-based model (`#pragma acc`
 470 `parallel loop`) occupies a similar conceptual niche to OpenMP’s target offload direc-
 471 tives (`#pragma omp target teams distribute`). Including both would provide dimin-
 472 ishing returns in programming-model diversity.
- 473 3. **Compiler availability:** OpenACC compilation requires the NVIDIA HPC SDK (`nvc`) or
 474 GCC with `-fopenacc`. This is a less universally available toolchain than CUDA (`nvcc`)
 475 or OpenMP (any modern C/C++ compiler), introducing a confounding variable between
 476 compiler availability and LLM translation capability.

477 B.5 OpenMP Target as Case Study

478 OpenMP target offload (`#pragma omp target`) occupies a middle ground between CPU OpenMP
 479 and GPU-native APIs. It uses the OpenMP directive model but targets GPU execution, requiring
 480 the programmer to specify data mapping (`#pragma omp target data map(...)`) and work dis-
 481 tribution (`#pragma omp teams distribute parallel for`) explicitly.

482 ParBench evaluates OpenMP target as a case study rather than a primary direction for three reasons:

- 483 1. **Compiler requirement:** OpenMP target compilation for NVIDIA GPUs requires the
 484 NVIDIA HPC compiler `nvc` (part of the NVIDIA HPC SDK 24.3), which is not universally
 485 available. Standard GCC/Clang OpenMP support targets only CPU threading. This intro-
 486 duces a confounding variable: a failed build may indicate compiler unavailability rather
 487 than translation quality.
- 488 2. **Limited benchmark coverage:** OpenMP target implementations are available for XS-
 489 Bench, RSBench, and a subset of HeCBench curated kernels (106 kernel-level co-
 490 occurrences with CUDA, as shown in Figure 5). Rodinia does not provide OpenMP target
 491 variants.
- 492 3. **Compilation model difference:** OpenMP target generates GPU offload code through the
 493 compiler, while CPU OpenMP generates threaded CPU code. A translation from CUDA
 494 to OpenMP target preserves GPU execution semantics but changes the syntax entirely; a
 495 translation from CUDA to CPU OpenMP changes both the execution model and the syntax.
 496 These are qualitatively different evaluation targets that should not be conflated in aggregate
 497 statistics.

498 The case study results for OpenMP target are reported separately in the evaluation (Section 5 of
 499 the main paper) to enable direct comparison with the primary CUDA/OpenMP/OpenCL directions
 500 without confounding the aggregate pass rates.

501 C Benchmark Kernel Survey

502 This appendix documents the systematic survey and multi-stage selection process that produced
 503 ParBench’s evaluation corpus. The main paper (Section 3) summarizes this process; here we provide
 504 the complete data, selection criteria, and exclusion rationale.

505 C.1 Full Repository Inventory

506 The survey identified 35 benchmark repositories spanning five categories of HPC benchmark soft-
507 ware. Figure 7 shows the distribution across categories.

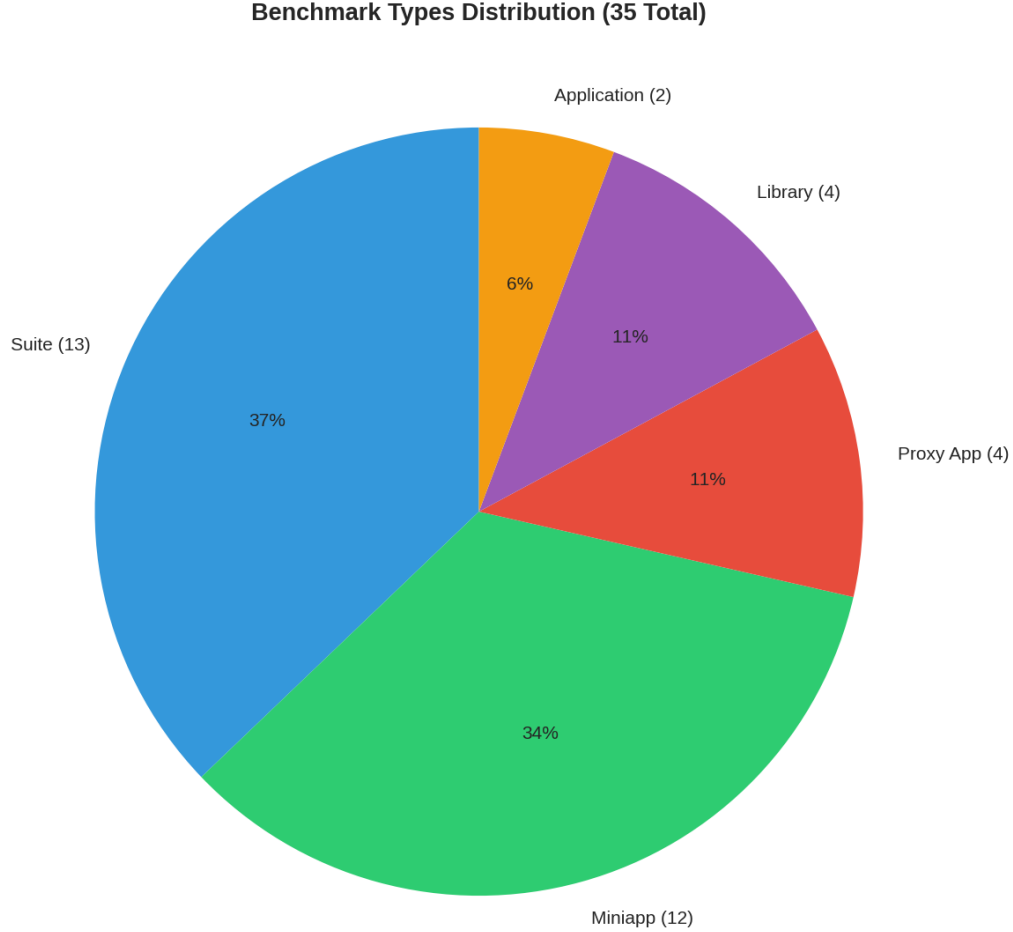


Figure 7: Distribution of benchmark types across the 35 surveyed repositories. Suites (13, 37%) and mini-applications (12, 34%) dominate, together accounting for 71% of the survey. Proxy applications (4, 11%), libraries (4, 11%), and full applications (2, 6%) complete the distribution. Suites and mini-applications provide the richest kernel-level material for translation evaluation because they contain multiple independent computational kernels with self-checking verification.

508 The relationship between benchmarks and APIs is not uniform: some repositories implement a single
509 API (e.g., SPEC OMP, PARSEC, LonestarGPU), while others provide implementations across 10+
510 APIs (BabelStream with 12, miniBUDE with 14).

511 Figure 8 quantifies the API breadth distribution across the surveyed benchmarks.

512 The combination of Figures 7–8 establishes that the benchmark landscape is dominated by a small
513 number of large suites with broad API coverage. This Pareto distribution motivated the selec-
514 tion strategy: focus on the richest suites (HeCBench, Rodinia) supplemented by targeted mini-
515 applications (XS Bench, RS Bench, mixbench) to ensure domain diversity.

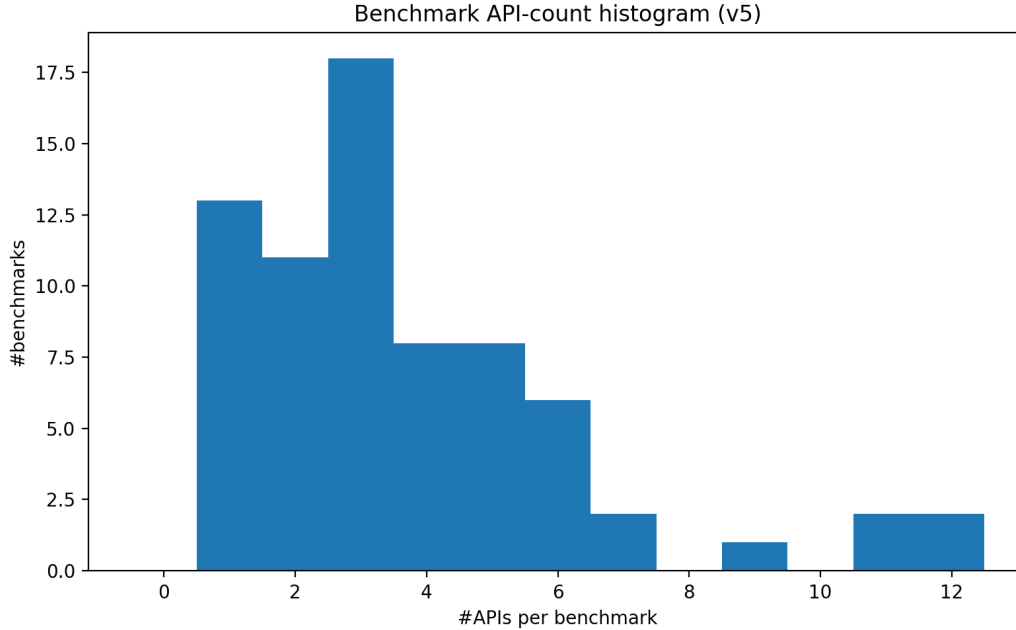


Figure 8: Distribution of API breadth (number of parallel APIs supported) across surveyed benchmarks. The modal benchmark supports 3 APIs, with a peak of approximately 18 benchmarks in the 3-API bin. A long tail extends to 12+ APIs, occupied by BabelStream and miniBUDE. Approximately 13 benchmarks support only 1–2 APIs (single-paradigm implementations). Benchmarks supporting 3 or more APIs are the primary candidates for multi-direction translation evaluation.

516 C.2 Quality Tier Classification

517 All 35 surveyed repositories were classified into two quality tiers based on three criteria: build
518 documentation, verification capability, and maintenance status.

519 **Tier A criteria** (all three required):

- 520 • Documented build process (Makefile, CMake, or equivalent with clear instructions)
- 521 • Automated verification: self-checking output (print PASS/FAIL, compute checksum, com-
522 pare against reference) or reference output file comparison
- 523 • Active maintenance: commits within 2 years of the survey date, or stable release with no
524 known build failures on modern toolchains

525 **Tier B criteria** (any one sufficient for downgrade):

- 526 • Partial verification only (e.g., visual output inspection, manual comparison)
- 527 • Limited API coverage (single API, reducing cross-translation utility)
- 528 • Unmaintained but still buildable (no commits in 2+ years, but Makefiles still produce work-
529 ing binaries)

530 Tier B repositories were not excluded from the survey but were deprioritized in kernel selection.
531 Their kernels were considered only when no Tier A alternative existed for a given computational
532 domain.

533 C.3 Candidate Ranking and HeCBench Selection Funnel

534 The selection of individual kernels for ParBench’s evaluation corpus required balancing two com-
535 peting objectives: *kernel richness* (maximizing the number of independent evaluation instances) and

536 *API breadth* (maximizing the number of translation directions per kernel). Figure 9 plots these two
537 dimensions for all surveyed repositories.

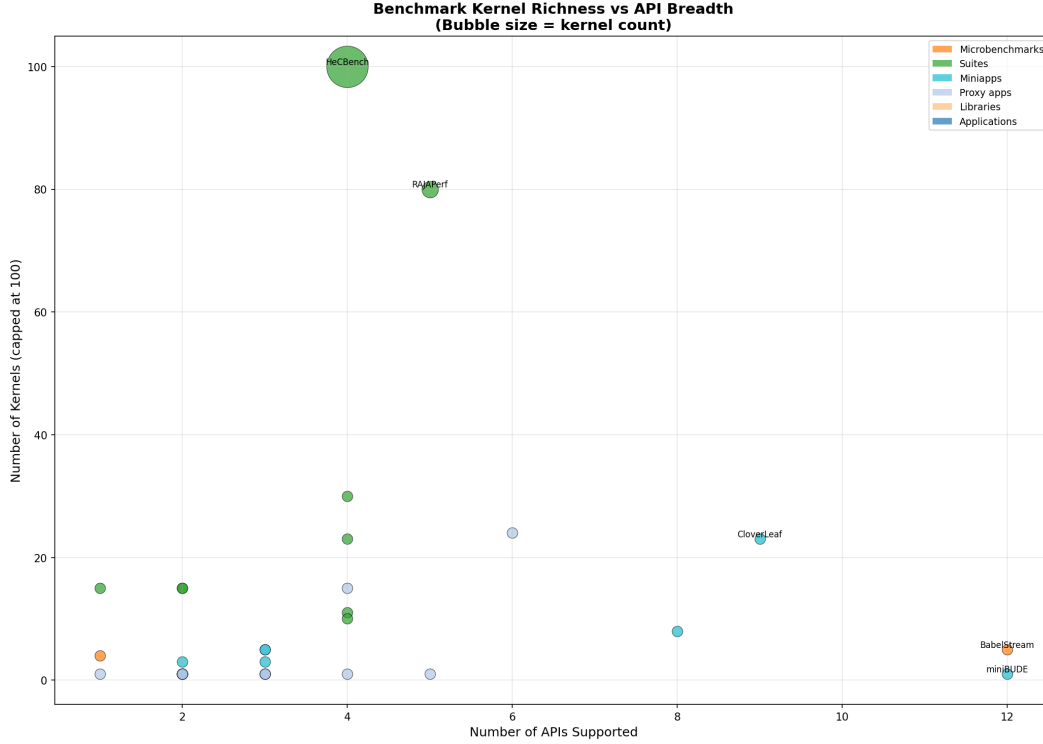


Figure 9: Benchmark kernel richness (y-axis, capped at 100 for readability) vs. API breadth (x-axis, number of APIs supported). Bubble size proportional to total kernel count; color indicates benchmark type. HeCBench (516 kernels, 4 APIs) dominates the upper region. RAJAPerf (80 kernels, 5 APIs) is the second-largest. CloverLeaf (23 kernels, 9 APIs) and BabelStream (5 kernels, 12 APIs) occupy the high-API-breadth region but with far fewer kernels. The optimal candidates for corpus construction lie in the high-kernel, moderate-API quadrant (upper-center).

538 Figure 10 presents the top candidates ranked by their combined kernel-count and API-breadth scores,
539 revealing the fundamental tradeoff in corpus construction.

540 HeCBench Selection Funnel

541 HeCBench’s 516 kernels were filtered through a multi-stage funnel to identify the curated evaluation
542 set:

- 543 • **Stage 1 (API filter):** 516 total kernels in the HeCBench repository. Of these, 272 provide
544 implementations in all 4 primary APIs (CUDA, HIP, SYCL, OpenMP) with Makefiles and
545 self-checking output. The remaining 244 lack one or more API variants, Makefiles in the
546 CUDA variant, or self-checking output patterns.
- 547 • **Stage 2 (Complexity filter):** The 272 candidates were filtered for evaluation tractability.
548 Kernels with more than 15 source files or fewer than 2 were excluded (the former are too
549 complex for single-prompt translation; the latter are trivial boilerplate). Kernels requiring
550 external input data files were also excluded to ensure self-contained evaluation.
- 551 • **Stage 3 (Domain diversity):** From the complexity-filtered set, 60 kernels were selected
552 to maximize domain diversity across computational categories: linear algebra, stencil com-
553 putation, graph algorithms, machine learning, physics simulation, sorting, reduction, scan,
554 image processing, and cryptography.

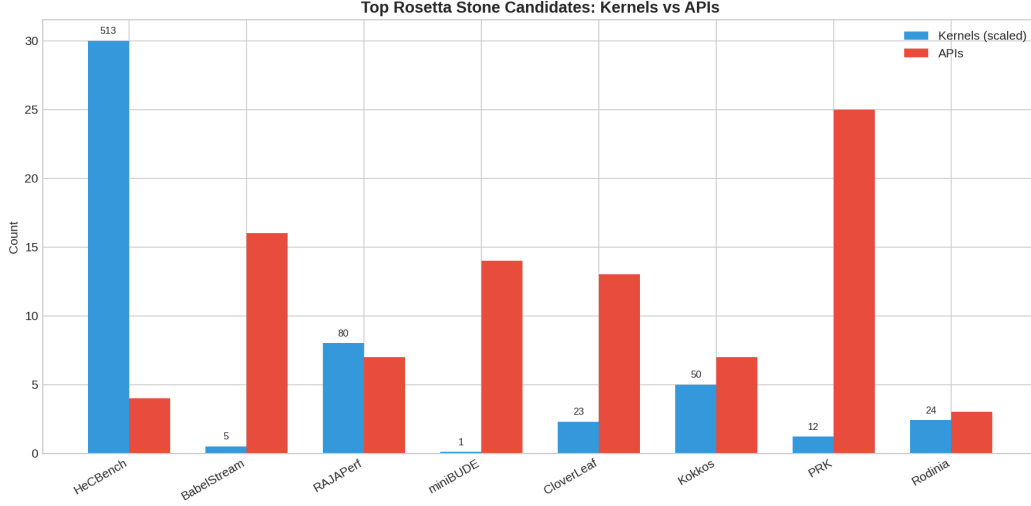


Figure 10: Top “Rosetta Stone” candidates ranked by kernel count (blue bars, scaled for readability; true counts annotated above) and API breadth (red bars). HeCBench provides the largest kernel pool (516 kernels) but with only 4 primary APIs (CUDA, HIP, SYCL, OpenMP). BabelStream provides the broadest API coverage (12 APIs) but with only 5 kernels. RAJAProf (80 kernels, 7 APIs) offers a balanced middle ground. Rodinia (24 kernels, 3 primary APIs) provides established, well-understood benchmarks with strong community recognition.

555 • **Stage 4 (Curation):** The 60-kernel working set was narrowed to a final curated set of 20
556 kernels based on manual inspection of build reliability, output determinism, and domain
557 representativeness. Of these, 10 kernels were included in the current ParBench evaluation
558 corpus, with the remaining 10 reserved for future expansion.

559 The final 20 curated HeCBench kernels span diverse computational domains and algorithmic pat-
560 terns, ensuring that the evaluation corpus tests LLM translation capability across a representative
561 sample of real HPC workloads.

562 C.4 Rodinia Kernel Inventory

563 Rodinia provides the foundation of ParBench’s evaluation corpus. As one of the most widely cited
564 HPC benchmark suites (originally published at IISWC 2009 Che et al. [2009]), it offers strong com-
565 munity recognition and well-understood computational characteristics. ParBench uses 22 Rodinia
566 kernels across three APIs (CUDA, OpenMP, OpenCL), yielding 60 total specs (some kernels lack
567 one or more API variants).

568 Of these 60 specs:

- 569 • **54 TRUE PASS:** Verified correct output with exit code 0 and stdout pattern matching using
570 conjunction verification (both conditions must hold).
- 571 • **6 KNOWN_FAIL:** Excluded from evaluation batches due to pre-existing build or runtime
572 failures unrelated to LLM translation. These include 2 kernels using deprecated CUDA
573 texture<> references removed in CUDA 12 (kmeans-cuda, mummergpu-cuda), 1 kernel
574 requiring OpenGL dependencies (hybridsort-cuda), 1 kernel with CUDA API depen-
575 dencies in its OpenMP variant (mummergpu-omp), and 2 kernels with pre-existing OpenCL
576 runtime issues (nn-openc1, kmeans-openc1).

577 C.5 XSbench, RSbench, and mixbench Profiles

578 Three additional benchmark suites complement Rodinia and HeCBench in the evaluation corpus:

579 **XSbench** (4 specs: CUDA, OpenMP, OpenCL, OpenMP Target): A Monte Carlo neutron cross-
 580 section lookup proxy application from Argonne National Laboratory. XSbench implements a hash-
 581 based unionized energy grid algorithm that is representative of nuclear reactor simulation workloads.
 582 All 4 API variants pass baseline verification. The OpenMP variant produces a history-based check-
 583 sum (941535); the CUDA, OpenCL, and OpenMP Target variants produce an event-based checksum
 584 (945990). This asymmetry reflects algorithmic differences between the CPU and GPU implementa-
 585 tions rather than correctness issues.

586 **RSbench** (4 specs: CUDA, OpenMP, OpenCL, OpenMP Target): A simplified variant of XSbench
 587 that isolates the multipole cross-section lookup kernel. RSbench was added to ParBench to enable
 588 controlled comparison with XSbench on the same algorithmic family at different complexity levels.

589 **mixbench** (3 specs: CUDA, OpenMP, OpenCL): A micro-benchmark designed to measure GPU
 590 arithmetic throughput at varying compute-to-memory ratios. mixbench exercises the roofline bound-
 591 ary, making it valuable for evaluating whether LLM translations preserve performance-critical arith-
 592 metic intensity.

593 C.6 Exclusion Log and Verification Methods

594 Figure 11 groups the surveyed benchmarks by their verification approach, illustrating the diversity
 595 of correctness checking methods across the survey and the basis for exclusion decisions.

596 Repository-Level Exclusions

597 Five of the initial 40 candidate repositories were excluded during the survey phase:

- 598 • **Download failures** (2): Repository URLs were dead or access-restricted at the time of
 599 survey.
- 600 • **Insufficient documentation** (2): No build instructions, no Makefiles, or build system re-
 601 quired proprietary dependencies not available on the evaluation platform.
- 602 • **Duplicate content** (1): One repository was a fork of an already-surveyed suite with no
 603 additional API variants.

604 Kernel-Level Exclusions

605 Within the selected suites, individual kernels were excluded for the following reasons:

- 606 • **Missing API variants**: Kernels available in only one API (no cross-translation possible).
- 607 • **No self-checking output**: Kernels that produce graphical output, require manual inspec-
 608 tion, or have no deterministic expected output.
- 609 • **Excessive complexity**: Kernels requiring more than 15 source files, external data files not
 610 included in the repository, or runtime dependencies (e.g., OpenGL, MPI multi-node) not
 611 available on the single-GPU evaluation platform.
- 612 • **Non-deterministic output**: Kernels whose output depends on execution order, random
 613 seeds without fixed initialization, or floating-point non-associativity beyond the tolerance
 614 of the verification method.

615 The complete exclusion log with per-kernel justifications is maintained in ParBench’s specification
 616 metadata (specs/*.json) and the project’s known-issues documentation.

617 D Extended Related Work

618 The main text positions ParBench through the evaluation gap it addresses. This appendix records
 619 the longer comparison across granularity, verification rigor, and robustness testing.

620 ParEval Nichols et al. [2024] evaluates parallel code generation from natural language descriptions.
 621 It establishes that LLMs can synthesize parallel code in some settings, but source-to-source transla-
 622 tion requires preserving existing program structure and API semantics rather than inventing a solu-
 623 tion from a description. ParEval-Repo Davis et al. [2025] evaluates full repositories and therefore

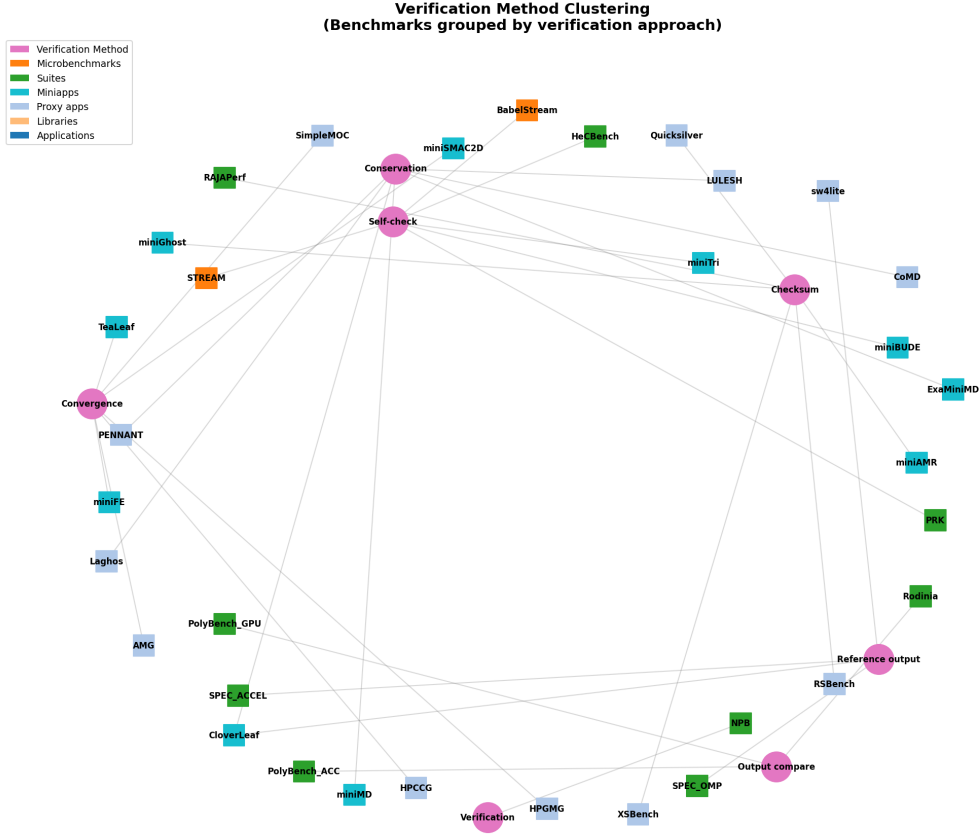


Figure 11: Benchmarks grouped by verification method. Large colored nodes represent verification approaches: checksum comparison, convergence testing, reference output file comparison, output-to-standard comparison, and self-validation. Benchmark nodes (smaller, colored by type) are connected to their verification method(s). Benchmarks without automated verification appear as weakly connected or isolated nodes and were excluded or downgraded to Tier B. The checksum and reference output clusters contain the majority of Tier A benchmarks.

Table 7: Related work comparison along granularity and verification axes. ParBench combines kernel-level HPC translation, build-run-verify correctness, and augmentation-based robustness evaluation.

Paper	Venue	Granularity	Verification	HPC parallel	Augmentation	Scale
HumanEval Chen et al. [2021]	arXiv’21	Function	Unit tests	–	–	164 functions
SWE-bench Jimenez et al. [2024]	ICLR’24	Repository	Test suite	–	–	2,294 issues
TransCoder Rozière et al. [2020]	NeurIPS’20	Function	Unit tests	–	–	852 functions
CodeRosetta TehraniJamsaz et al. [2024]	NeurIPS’24	Function	BLEU + compile	Yes	–	C++/CUDA corpus
OMPify Kadosh et al. [2023]	IWOMP’23	Loop	Pragma accuracy	Partial	–	54K snippets
LASSI Dearing et al. [2024]	CLUSTER’24	Kernel	Build+run+verify	Yes	–	10 HeCBench
ParEval Nichols et al. [2024]	HPDC’24	Task	Correctness check	Yes	–	420 tasks
ParEval-Repo Davis et al. [2025]	ICPP’25	Repository	Build + functional	Yes	–	6 apps
HPC-Coder-V2 Chaturvedi et al. [2024]	arXiv’24	Task	pass@ <i>k</i>	Yes	–	ParEval tasks
TRACY Gong et al. [2025]	arXiv’25	Function	Tests + efficiency	–	Stress tests	1,000 tasks
ParBench	–	Kernel	Build+run+verify	Yes	L0–L4	96 specs

captures realistic integration work, but it also shows that build-system generation can dominate the result. ParBench occupies the kernel level between these extremes.

LASSI Dearing et al. [2024] is the closest prior system in granularity and verification style. It evaluates an agentic self-correction pipeline on 10 HeCBench kernels and two API directions, reporting high pass rates after iterative debugging. ParBench instead treats bounded self-repair as part of the measurement protocol and emphasizes raw translation capability, direction coverage, survey-grounded corpus construction, and augmentation robustness.

CodeRosetta TehraniJamsaz et al. [2024] and HPC-Coder-V2 Chaturvedi et al. [2024] show the value of specialized modeling and fine-tuning for HPC code. ParBench is complementary: it provides an execution-based substrate on which such models can be evaluated, including cases where compile or text-similarity metrics would miss functional errors. TRACY Gong et al. [2025] highlights efficiency as a separate axis from correctness; ParBench currently measures correctness only.

E Detailed Evaluation Findings

This appendix provides per-kernel and per-direction breakdowns of the evaluation results summarized in Section 5 of the main paper, including self-repair dynamics, augmentation transform statistics, the HeCBench selection funnel, and a detailed case study of multi-file translation consistency failures.

E.1 Self-Repair Transition Matrices

ParBench’s multi-attempt evaluation protocol allows LLMs to receive compiler/runtime error feedback and attempt corrections. Figure 12 presents the self-repair transition matrices, showing how initial failure modes map to final evaluation outcomes across all models and directions.

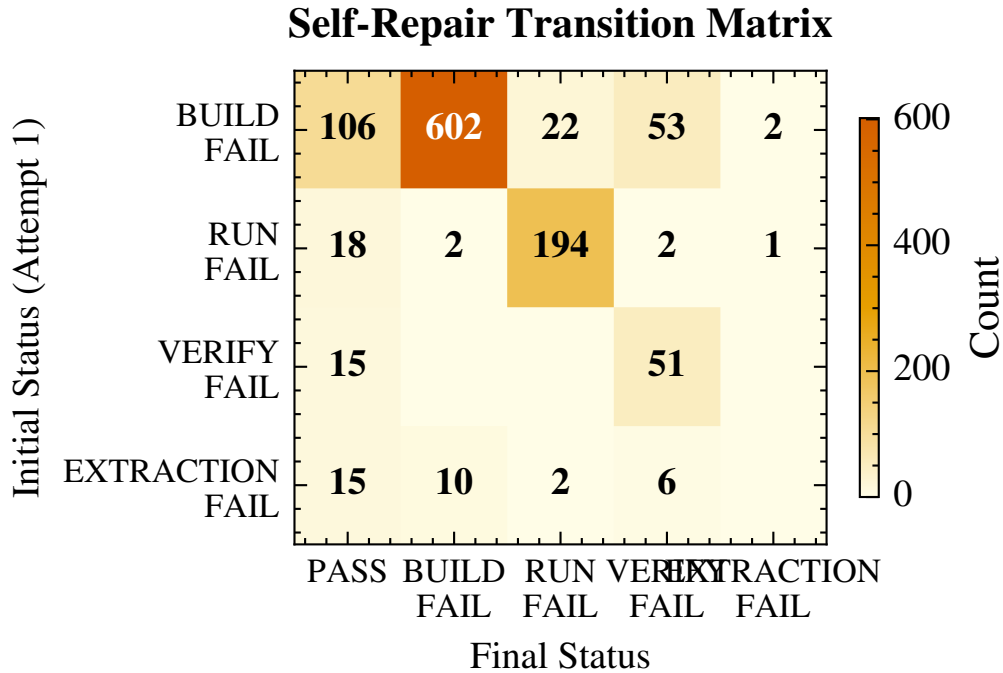


Figure 12: Self-repair transition matrix showing initial failure mode to final outcome counts across all translation directions. Rows represent the failure mode of the first attempt (BUILD_FAIL, RUN_FAIL, VERIFY_FAIL); columns represent the final evaluation outcome after up to 3 attempts. Off-diagonal entries indicate successful self-repair transitions.

645 The transition matrices reveal several key dynamics of LLM self-repair behavior:

- 646 • **Initial failure mode distribution:** The majority of first-attempt failures are BUILD_FAIL (missing headers, wrong API calls, syntax errors), followed by RUN_FAIL (segmentation faults, timeouts) and VERIFY_FAIL (incorrect output). This ordering reflects the pipeline’s sequential verification: build errors are caught before runtime errors, which are caught before output verification.
- 647
- 648
- 649
- 650
- 651 • **Repair rate asymmetry:** BUILD_FAIL has the highest repair rate, as compiler error messages provide actionable feedback (missing header name, undeclared identifier, type mismatch). RUN_FAIL has a moderate repair rate when error messages are informative (e.g., “segmentation fault at line X”) but low repair rate for opaque failures (e.g., timeouts with no diagnostic output). VERIFY_FAIL has the lowest repair rate, as output mismatches provide minimal diagnostic signal—the model receives “output did not match expected pattern” but lacks information about *which* computation diverged.
- 652
- 653
- 654
- 655
- 656
- 657
- 658 • **Token consumption:** Self-repair attempts consume additional tokens for both the error feedback prompt and the corrected translation. Evaluations requiring 3 attempts consume approximately $2.5\times$ the tokens of first-attempt successes. However, the marginal cost of self-repair is low relative to the cost of a completely new translation, as the model can build on its previous attempt.
- 659
- 660
- 661
- 662
- 663 • **Attempt convergence:** Among evaluations that eventually pass, the median number of attempts is 2, indicating that most repairable failures are resolved on the second try. Third-attempt successes are rare, suggesting diminishing returns from additional retry budget beyond 3.
- 664
- 665
- 666

667 E.2 Self-Repair Rate by Direction

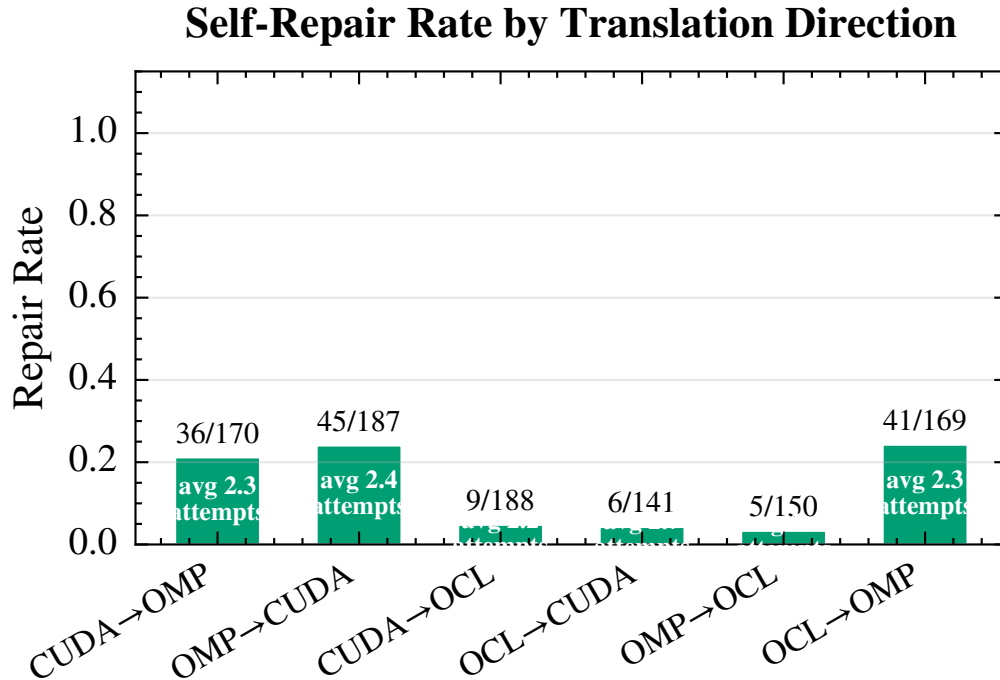


Figure 13: Self-repair rate by translation direction with attempt count distribution. Each bar shows the fraction of initially-failing evaluations that are rescued by subsequent attempts, broken down by the number of attempts required. Directions are ordered by repair rate.

The repair rate varies significantly by translation direction (Figure 13). Paradigm-preserving directions (CUDA-to-OpenMP, OpenMP-to-CUDA) show higher repair rates than cross-paradigm directions involving OpenCL, consistent with the structural similarity of CUDA and OpenMP parallel patterns despite their different execution models.

This directional asymmetry in repair effectiveness has practical implications for LLM-based translation tooling. A self-repair budget of 3 attempts is sufficient to capture most recoverable failures in CUDA $\leftrightarrow$ OpenMP directions, but provides diminishing returns for OpenCL targets where the failure modes (missing address space qualifiers, C++ to C99 decomposition) are structurally harder to diagnose from compiler error messages alone.

Preliminary pass@ k experiments further illuminate the self-repair mechanism. Comparing the primary campaign (temperature = 0.0, 3 retries with compiler feedback) against stochastic sampling (temperature = 0.7, 1 attempt, 3 independent samples) on CUDA-to-OpenMP reveals a $2.3\times$ improvement from self-repair with greedy decoding (63.6% vs. 27.3% pass rate). Notably, 7 of 11 kernel triplets produce identical outcomes across all 3 stochastic samples, indicating that sampling diversity alone is insufficient to overcome systematic translation errors. Self-repair with compiler error feedback is the dominant success factor, not stochastic exploration of the output distribution.

E.3 Augmentation Transform Frequency Per Kernel

ParBench’s code augmentation pipeline applies six AST-level transforms at four intensity levels (L1–L4). The transforms do not apply uniformly: some kernels have many candidate sites for a given transform, while others have few or none. Figure 14 presents the per-kernel, per-transform frequency heatmap.

Aggregate transform frequencies across the corpus reveal a clear dominance hierarchy:

- **SwapCondition:** 162 candidate sites (most common). Targets `if/else` branches and ternary expressions. Its prevalence reflects the ubiquity of conditional logic in HPC kernels—boundary checks, loop guards, and branch-based algorithm selection all contribute candidate sites.
- **ArithmeticTransform:** 69 candidate sites. Applies commutative rewriting of arithmetic expressions (e.g., $a + b \rightarrow b + a$, $a \times (b + c) \rightarrow a \times b + a \times c$). Less common than SwapCondition because many arithmetic expressions in HPC code involve non-commutative operations (subtraction, division) or are too simple (single-operand) to transform meaningfully.
- **ChangeNames:** 55 candidate sites. Performs variable and parameter renaming while preserving semantic equivalence. The count is bounded by the number of local variables and parameters per kernel, which is typically 10–30 for Rodinia kernels.
- **TypedefExpansion:** 7 candidate sites. Inlines typedef aliases to their underlying types. Rare because most Rodinia kernels use primitive types directly rather than defining aliases.
- **PointerArithmeticToArrayIndex:** 6 candidate sites. Rewrites `*(ptr + i)` to `ptr[i]` and vice versa. Limited by the prevalence of pointer arithmetic in the corpus—most kernels use array indexing by default.
- **ChangeFunctionNames:** 2 candidate sites (least common). Limited by the function definition count in single-kernel files, which typically contain only 1–3 user-defined functions.

The level-invariance property (54/60 Rodinia specs pass at all levels L1–L4) confirms that augmentation introduces no new correctness failures. This is expected: all transforms preserve semantic equivalence by construction, and the augmentation unit tests (`c_augmentation/test_transforms.py`, 15 tests) verify this property for each transform in isolation. From an evaluation perspective, augmentation levels serve as a robustness probe—demonstrating that LLM translation quality is stable under semantics-preserving source perturbations—rather than as independent samples that inflate the effective sample size.

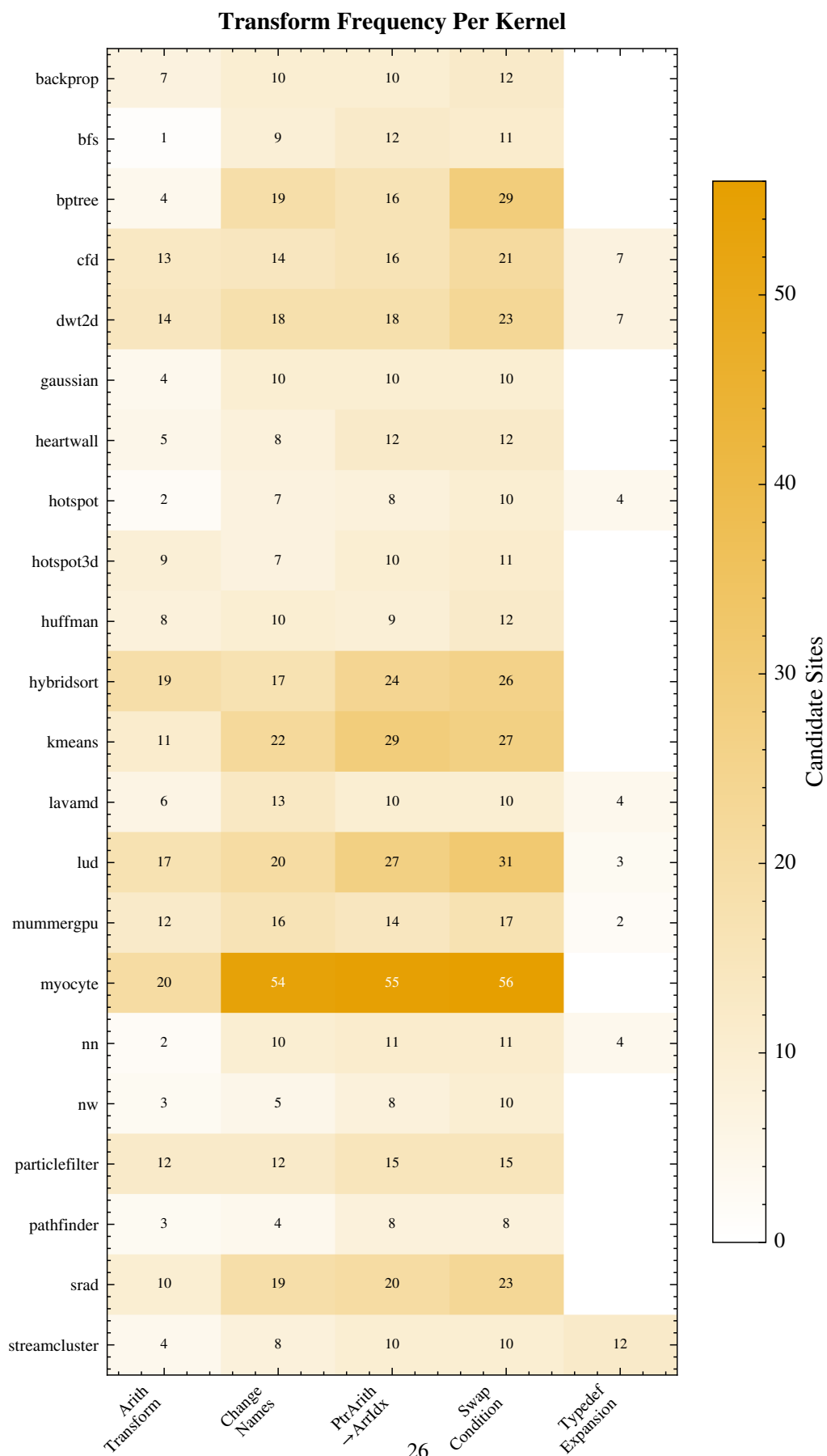


Figure 14: Transform frequency heatmap showing candidate sites per kernel per transform type.

HeCBench Kernel Selection Pipeline

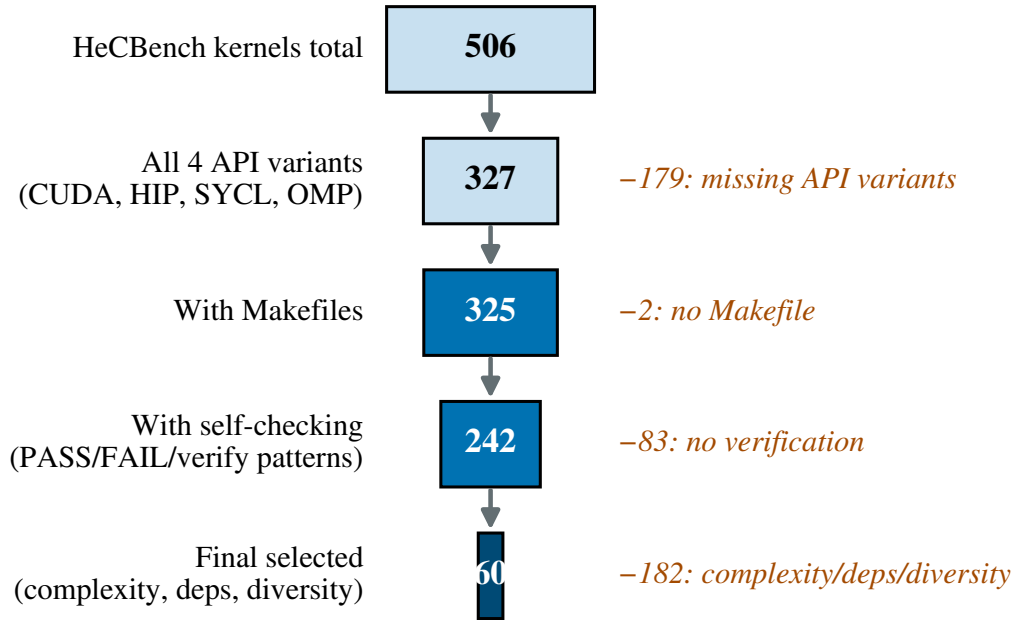


Figure 15: HeCBench kernel selection funnel. Starting from 513 kernels spanning CUDA, HIP, SYCL, and OpenMP, successive filters for API completeness (327 with all 4 variants), build infrastructure (325), self-checking output patterns (242), complexity bounds, and domain diversity yield the 60-kernel working set and final 20-kernel curated evaluation corpus.

E.4 HeCBench Kernel Selection Funnel

HeCBench is the largest source of multi-API kernel implementations in the benchmark landscape survey, with 513 kernels spanning CUDA, HIP, SYCL, and OpenMP (Figure 15). Its scale necessitated a structured selection funnel to identify kernels suitable for automated evaluation:

1. **API completeness** (513 $\rightarrow$ 327): Retained only kernels providing implementations in all 4 primary APIs. Kernels missing one or more API variants cannot support the full matrix of cross-API translation directions that ParBench evaluates.
2. **Build infrastructure** (327 $\rightarrow$ 325): Excluded 2 kernels whose CUDA variants lacked Makefiles or had broken build configurations. All retained kernels build successfully with `make` on the evaluation platform (NVIDIA HPC SDK 24.3, CUDA 12.3).
3. **Self-checking output** (325 $\rightarrow$ 242): Excluded kernels without deterministic, machine-parseable output. Retained kernels print checksums, PASS/FAIL indicators, or numerical results that can be matched against regular expression patterns for automated verification.
4. **Complexity bounds** (242 $\rightarrow$ 60): Excluded kernels with more than 15 source files (too complex for single-prompt LLM translation) or fewer than 2 source files (trivial boilerplate). Also excluded kernels requiring external input data files not bundled with the repository.
5. **Domain diversity and curation** (60 $\rightarrow$ 20): Manual inspection for build reliability, output determinism, and domain representativeness. The final 20 kernels span linear algebra, stencil computation, graph algorithms, machine learning, physics simulation, sorting, reduction, scan, image processing, and cryptography. Ten kernels are included in the current evaluation corpus; ten are reserved for future expansion.

This funnel illustrates a general challenge in constructing LLM code translation benchmarks: the vast majority of available HPC kernels are unsuitable for automated evaluation due to missing API variants, non-deterministic output, or excessive complexity. Only 3.9% of HeCBench’s 513 kernels (20/513) survive the full selection pipeline, highlighting the gap between “available parallel code” and “evaluable parallel code.”

E.5 Case Study: Multi-File Translation Consistency Failure

Full-program translations—where the LLM must rewrite both kernel code and host code—expose a class of errors absent from kernel-only translations. We document this failure class through a detailed case study of the `rodinia-lavamd` kernel translated from OpenCL to CUDA.

Observation

Across all 5 augmentation levels (L0–L4), the model (Qwen 3.5 397B) translates OpenCL kernel computational patterns to CUDA correctly (`__global__`, `blockIdx.x`, `threadIdx.x`, `__syncthreads()`) but **fails to produce a compilable multi-file CUDA program** in all 15 attempts (5 levels $\times$ 3 retries). The failure modes vary across augmentation levels—this is not a single consistent bug, but a family of related multi-file consistency errors.

Per-Level Error Taxonomy

Table 8: Multi-file translation error taxonomy for `lavamd` OpenCL-to-CUDA across augmentation levels. The failure mode varies by level, demonstrating that augmentation perturbs *which* errors the model makes without changing the outcome (BUILD_FAIL at all levels).

Level	Naming Correct?	Primary Failure Mode
L0	No	Identifier renaming failure (retained <code>kernel_gpu_openc1</code>)
L1	Yes	Phantom dependencies (<code>timing.h</code> , <code>checkCUDAError</code>)
L2	No	Identifier renaming failure (retained source name)
L3	Yes	Undefined helpers (<code>setdevice</code> , <code>checkCUDAError</code>)
L4	Yes	Undefined helpers + missing kernel launch syntax

Table 8 reveals that the “semantic renaming failure” (keeping `kernel_gpu_openc1` in CUDA code) only occurs at L0 and L2. At L1, L3, and L4, the model correctly renames the function but fails on different issues: phantom include paths, undefined helper functions, and missing `<<>>` kernel launch syntax.

Retry Behavior

All 5 levels share the same first-attempt error: the model hallucinates a non-existent header (`timing.h`). Subsequent attempts diverge by level, introducing *new* error categories without converging toward a fix. This non-convergent retry pattern—where each attempt produces qualitatively different errors rather than incrementally correcting the previous attempt—demonstrates a fundamental limitation of compiler-error-feedback-based self-repair for multi-file consistency errors. The error messages are locally informative (“undefined reference to `checkCUDAError`”) but do not convey the systemic issue: the model must generate a coherent program scaffold, not just fix individual compilation errors.

Implications

This case study identifies four distinct error subcategories within the “multi-file translation consistency failure” class:

1. **Phantom dependencies:** The model hallucinates include files (`timing.h`) or helper functions (`checkCUDAError`, `setdevice`) that exist in the model’s training data but not in the target program’s file structure.
2. **Cross-file identifier inconsistency:** The kernel file uses the target API’s naming convention (`kernel_gpu_cuda`) while the wrapper file retains the source API’s name (`kernel_gpu_opengl`), causing linker errors.
3. **Missing host-side launch syntax:** The wrapper calls the kernel as a regular C function (`kernel_gpu_cuda(...)`) rather than using CUDA kernel launch syntax (`kernel_gpu_cuda<<grid, block>>>(...)`).
4. **Scaffold generation failure:** The model cannot synthesize the boilerplate infrastructure (device selection, error checking, memory allocation/deallocation) needed for a complete CUDA program, instead relying on non-existent helper functions.

These findings suggest that a hybrid pipeline—combining LLM translation for computational kernels with template-based scaffold generation for host infrastructure—may outperform end-to-end LLM translation for multi-file programs. However, the varying failure modes across augmentation levels indicate that a simple post-processing pass (e.g., find-and-replace renaming) would only address subcategory 2, leaving subcategories 1, 3, and 4 unresolved.

F Supplementary Tables and Figures

This appendix contains detailed evidence tables and figures referenced from the main text. Floats are ordered by their original section appearance.

Table 9: Programming model characteristics of the three primary translation APIs. These differences define the structural transformation challenges that LLM-based translation must address.

Property	CUDA	OpenMP	OpenCL
Execution model	SIMT (warps of 32 threads)	Fork-join thread teams	NDRange work-groups
Memory management	Explicit device allocation and copies	Implicit shared CPU memory	Explicit buffer objects and command queues
Kernel specification	Compiled with host code	Pragma directives over loops	Runtime string compilation
Source structure	Single or multi-file (51% multi-file in corpus)	Single file (directives annotate sequential code)	Separate host program + .cl kernel files
Thread indexing	Block/thread indices	Loop variable or thread query	Global/local ID queries

Table 10: Model configurations.

Model	Provider	Arch.	Parameters	Temp.
Qwen 3.5 397B-A17B	Together AI	MoE	397B / 17B active	0
GPT-4.1 mini	Azure OpenAI	undisclosed	undisclosed	0

Table 11: Hardware and software configuration.

Component	Qwen (primary)	GPT-4.1 mini (collaborator)
GPU	RTX 4070 (12 GB)	NVIDIA A100 (40 GB HBM2)
CPU	Ryzen 9 7900X	AMD EPYC 7543P (32 cores)
OS	Ubuntu 24.04 LTS	Ubuntu 24.04 LTS
CUDA toolkit	HPC SDK 24.3	CUDA-compatible ALCF software stack
C/C++ compiler	GCC 12.4	matching campaign configuration
OpenCL runtime	NVIDIA HPC SDK	NVIDIA runtime where applicable
Python	3.12.3	matching campaign configuration

Table 12: Self-repair failure mode transitions (all tasks, Qwen 3.5). Rows: initial failure mode. Columns: final outcome after up to 3 retries.

Initial Failure	→PASS	Persist.	Regr.	Total	Repair Rate
BUILD_FAIL	72	274	–	346	20.8% [16.9%, 25.4%]
RUN_FAIL	17	131	–	148	11.5% [7.3%, 17.6%]
VERIFY_FAIL	8	20	–	28	28.6% [15.2%, 47.1%]
EXTR_FAIL	15	12	–	27	55.6% [37.3%, 72.4%]
UNKNOWN	0	1	–	1	0.0%

Table 13: Self-repair effectiveness for the Qwen 3.5 primary campaign.

Metric	Value	Interpretation
Total tasks	710	all directions and augmentation levels
First-attempt PASS	160 (22.5%)	deterministic translation before repair
Repaired after feedback	112 (15.8%)	initially failing tasks recovered
Total PASS	272 (38.3%)	final greedy pass rate
Relative improvement	+70.0%	$(272 - 160)/160$
Persistent fail	392	no successful repair
Partial repair	39	failure category improved but not PASS
Regression	7 (1.0%)	initially passing task later failed

Table 14: LLM pass rates across augmentation levels. All directions ($n = 142$ per level) and balanced CUDA-to-OpenMP subset ($n = 24$ per level).

Level	Qwen 3.5 (all dirs, $n=142$)	Qwen 3.5 (C→OMP, $n=24$)	GPT-4.1 mini
L0	40.1% [32.4%, 48.4%]	66.7% [46.7%, 82.0%]	28.1% [20.6%, 36.9%]
L1	37.3% [29.8%, 45.5%]	58.3% [38.8%, 75.5%]	28.2% [20.6%, 37.2%]
L2	40.1% [32.4%, 48.4%]	70.8% [50.8%, 85.1%]	30.3% [22.4%, 39.5%]
L3	39.4% [31.8%, 47.6%]	58.3% [38.8%, 75.5%]	29.4% [21.6%, 38.5%]
L4	34.5% [27.2%, 42.6%]	66.7% [46.7%, 82.0%]	30.3% [22.4%, 39.5%]

Table 15: Qwen 3.5 pass@ k results (L0, temperature=0.7, three independent samples, no self-repair).

Pairs	Greedy rate (T=0)	pass@1 (T=0.7)	pass@3 (T=0.7)	Hard Fail	Always Pass
142 direction–kernel pairs	38.3% (primary campaign)	19.7% (macro)	27.5% (macro)	103/142 (72.5%)	17/142 (12.0%)

Table 16: Statistical summary. All confidence intervals are Wilson 95% CIs.

Test	Statistic	p-value	Effect Size	Interpretation
Model comparison (chi-squared)	$\chi^2 = 7.83, df = 1$	$p = 0.005$	Cohen’s $h = 0.16$	Significant difference; small effect
Qwen augmentation (Cochran–Armitage)	$z = 0.0$	$p = 1.0$	–	No monotonic trend; underpowered for moderate effects
Direction: cuda-omp (McNemar)	$n = 24$, exact	$p = 0.6875$	Cohen’s $h = -0.17$	No significant asymmetry ($\alpha = 0.0125$)
Direction: cuda-opencl (McNemar)	$n = 20$, exact	$p = 0.6875$	Cohen’s $h = 0.28$	No significant asymmetry ($\alpha = 0.0125$)
Direction: omp-opencl (McNemar)	$n = 18$, exact	$p = 1.0$	Cohen’s $h = 0.12$	No significant asymmetry ($\alpha = 0.0125$)

Augmentation Robustness: Pass Rate across L0–L4
(CUDA→OpenMP, N=26 kernels/model, seed=42+L)

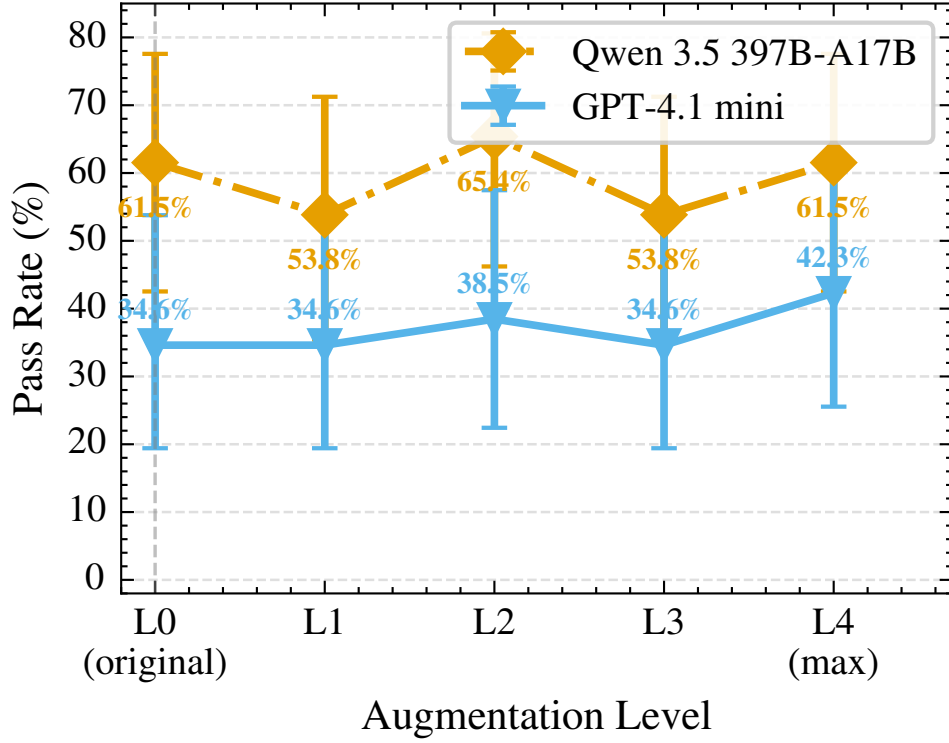


Figure 16: Augmentation robustness. X-axis: augmentation levels L0–L4. Y-axis: pass rate (%). Lines per model. Rodinia CUDA-to-OpenMP, seed=42.

G Translation Prompt Template

This appendix reproduces the full translation prompt template used by the evaluation pipeline. All prompts follow this structure; only the source code content, target API, and file lists vary between tasks. The anonymization protocol described in Section 2.2 is applied before prompt construction.

G.1 System Message

The system message is identical for all translation tasks:

```
You are a parallel programming expert
specializing in {src_api} to {tgt_api}
translation. Translate the provided source
code to {tgt_api}. Output ONLY the translated
code, no explanations. For each file, output
a markdown code fence with the filename on
the opening line:

'''{tgt_lang} filename={filename}
<code>
'''

Preserve the algorithm, I/O behavior, data
formats, and output format exactly. The
translated code must compile with the
```

Per-Kernel Augmentation Status (CUDA-to-OMP)

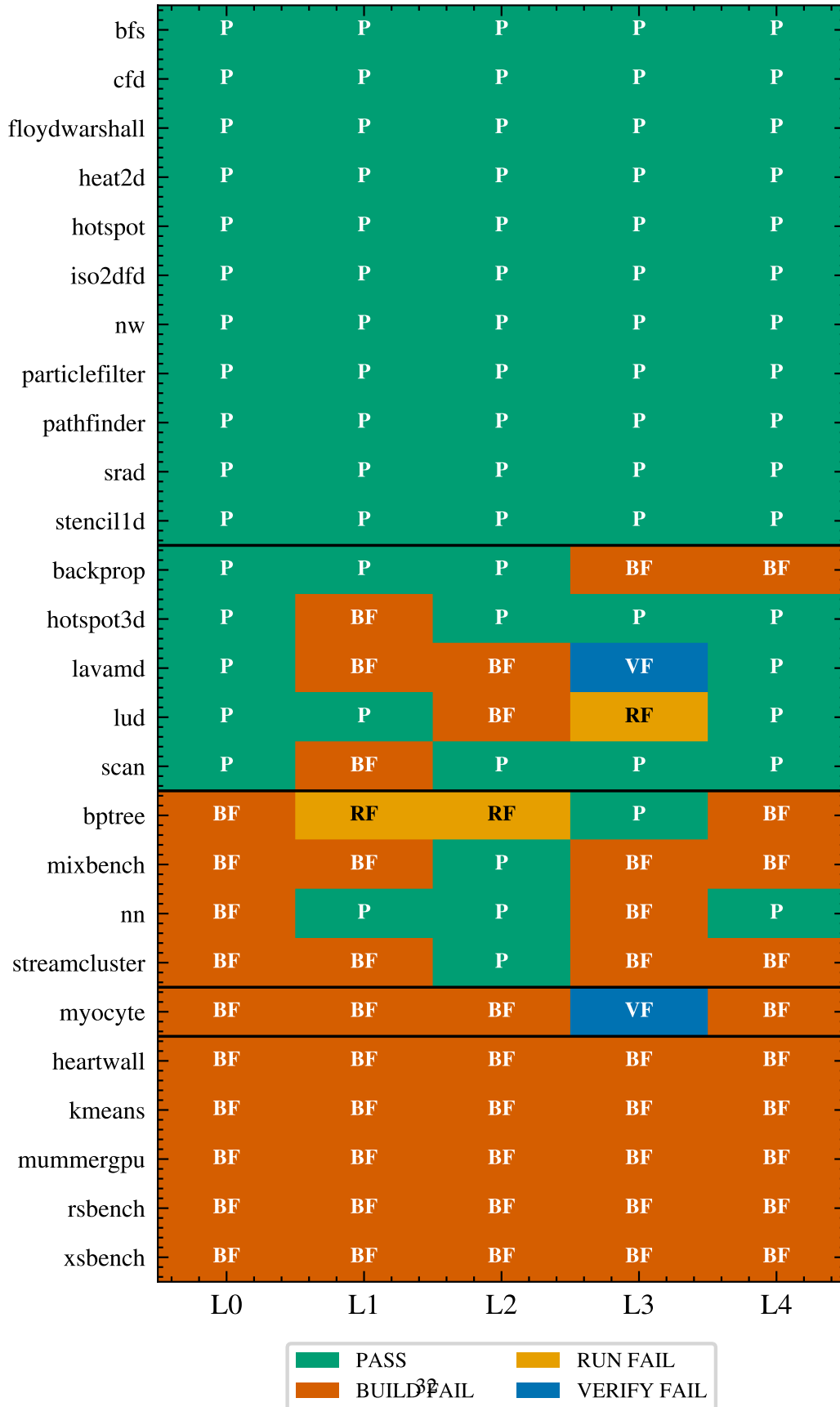


Figure 17: Per-kernel augmentation status across levels L0-L4 (CUDA-to-OMP, Owen 3.5). Cell

pass@k by Translation Direction (Qwen 3.5 397B-A17B)

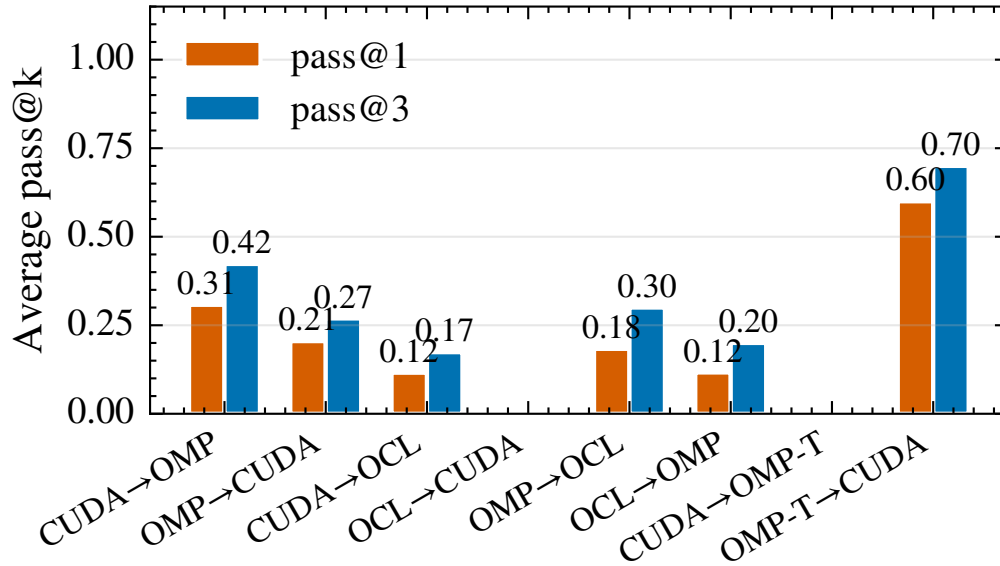


Figure 18: pass@k by translation direction.

815 provided build command.

Listing 4: System message template. {src_api} and {tgt_api} are replaced with API display names (e.g., “CUDA”, “OpenMP”). {tgt_lang} is the target language hint (e.g., “cpp”, “c”).

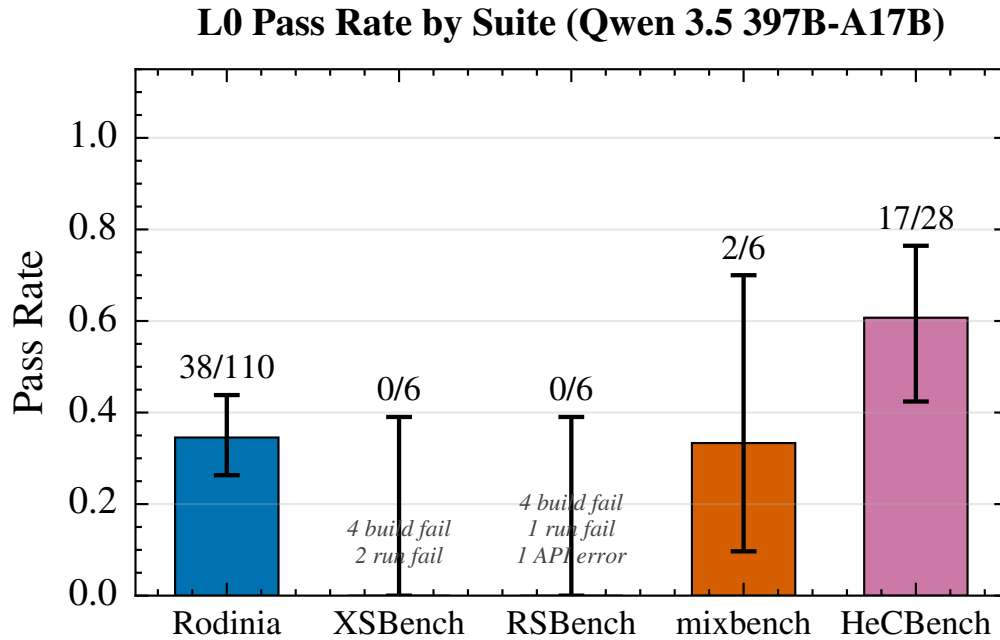
813 G.2 User Message Structure

814 The user message is assembled from the following sections, in order:

- 815 1. **Translation Task** — source API, target API. Kernel name and benchmark description are
816 *not* included (anonymization).
- 817 2. **Target Files to Produce** — list of anonymized target filenames (e.g., translated_0.cpp,
818 translated_1.cl).
- 819 3. **Build Command** — the anonymized compilation command in a code fence, so the LLM
820 can ensure API and flag compatibility.
- 821 4. **Source Code** — each source file presented as a numbered subsection (Source File 1,
822 Source File 2, ...) with comments stripped.
- 823 5. **Support / Header Files** (optional) — genericized header and support files from the source
824 directory, with an instruction to inline definitions rather than emit unresolvable #include
825 directives.
- 826 6. **Target Infrastructure Context** (optional) — non-kernel target files provided as read-only
827 reference so the LLM can match expected function signatures and data structures. Marked
828 “DO NOT MODIFY.”

829 G.3 Anonymization Details

830 Three anonymization steps are applied before prompt construction:



*XSBench/RSBench 0%: complex multi-file programs with inter-module dependencies — all translations failed (primarily build/linker errors).
HeCBench 61%: single-file kernels.*

Figure 19: Cross-suite pass rate comparison (L0, Qwen 3.5). Per-suite aggregate pass rates with Wilson 95% CIs across all 5 benchmark suites.

- 831 • **Comment stripping.** All C/C++ comments (`//` and `/* */`) are removed from source files
832 to prevent benchmark-specific annotations from leaking into the prompt.
- 833 • **Filename genericization.** Target filenames are replaced with `translated_0.ext`,
834 `translated_1.ext`, etc. Source files are labeled Source File 1, Source File 2. An
835 internal mapping restores original filenames when writing LLM output to disk.
- 836 • **Build command anonymization.** Kernel-specific identifiers are stripped from the build
837 command string.

838 H GPT-4.1 Mini Per-Model Figures

839 The following figures present GPT-4.1 mini evaluation results, complementing the Qwen 3.5 397B
840 figures in the main body. Each figure uses the same visualization format as its main-body counterpart
841 to enable direct visual comparison.

842 I Evaluation Cost Summary

Per-Kernel Translation Outcomes (L0, All Suites, GPT-4.1 mini)
35 kernels × 8 directions

	CUDA→OMP	OMP→CUDA	CUDA→OCL	OCL→CUDA	OMP→OCL	OCL→OMP	CUDA→OMP	OMP→CUDA
backprop	BF	BF	P	BF	P	VF	No OMP-Target tgt impl.	No OMP-Target src impl.
bfs	P	BF	P	BF	P	P	No OMP-Target tgt impl.	No OMP-Target src impl.
bptree	BF	BF	RF	BF	BF	BF	No OMP-Target tgt impl.	No OMP-Target src impl.
cfld	P	BF	BF	BF	BF	P	No OMP-Target tgt impl.	No OMP-Target src impl.
dwt2d	No OpenMP tgt impl.	No OpenMP src impl.	P	BF	No OpenMP src impl.	No OpenMP tgt impl.	No OMP-Target tgt impl.	No OMP-Target src impl.
gaussian	No OpenMP tgt impl.	No OpenMP src impl.	RF	P	No OpenMP src impl.	No OpenMP tgt impl.	No OMP-Target tgt impl.	No OMP-Target src impl.
heartwall	BF	BF	BF	—	BF	BF	No OMP-Target tgt impl.	No OMP-Target src impl.
hotspot	P	P	P	—	P	P	No OMP-Target tgt impl.	No OMP-Target src impl.
hotspot3d	P	BF	P	—	P	VF	No OMP-Target tgt impl.	No OMP-Target src impl.
huffman	No OpenMP tgt impl.	No OpenMP src impl.	No OpenCL tgt impl.	No OpenCL src impl.	No src or tgt impl.	No src or tgt impl.	No OMP-Target tgt impl.	No OMP-Target src impl.
hybridsort	No OpenMP tgt impl.	No OpenMP src impl.	RF	—	No OpenMP src impl.	No OpenMP tgt impl.	No OMP-Target tgt impl.	No OMP-Target src impl.
Rodinia								
kmeans	BF	BF	VF	—	VF	BF	No OMP-Target tgt impl.	No OMP-Target src impl.
lavamd	VF	BF	BF	—	BF	P	No OMP-Target tgt impl.	No OMP-Target src impl.
lud	BF	BF	BF	—	BF	P	No OMP-Target tgt impl.	No OMP-Target src impl.
mummergepu	BF	BF	No OpenCL tgt impl.	No OpenCL src impl.	No OpenCL tgt impl.	No OpenCL src impl.	No OMP-Target tgt impl.	No OMP-Target src impl.
myocyte	BF	BF	BF	—	BF	BF	No OMP-Target tgt impl.	No OMP-Target src impl.
nn	P	P	RF	—	RF	BF	No OMP-Target tgt impl.	No OMP-Target src impl.
nw	P	BF	RF	—	P	P	No OMP-Target tgt impl.	No OMP-Target src impl.
particlefilter	P	BF	VF	—	VF	P	No OMP-Target tgt impl.	No OMP-Target src impl.
pathfinder	P	P	P	—	P	BF	No OMP-Target tgt impl.	No OMP-Target src impl.
srld	P	BF	RF	—	RF	RF	No OMP-Target tgt impl.	No OMP-Target src impl.
streamcluster	BF	BF	RF	—	RF	BF	No OMP-Target tgt impl.	No OMP-Target src impl.
XSbench bench	BF	—	BF	—	BF	BF	—	—
RSBench bench	BF	—	BF	—	BF	BF	—	—
mixbench bench	BF	—	BF	—	BF	BF	No OMP-Target tgt impl.	No OMP-Target src impl.
convolution1d	No OpenMP tgt impl.	No OpenMP src impl.	No OpenCL tgt impl.	No OpenCL src impl.	No src or tgt impl.	No src or tgt impl.	BF	—
floydwarshall	BF	BF	No OpenCL tgt impl.	No OpenCL src impl.	No OpenCL tgt impl.	No OpenCL src impl.	BF	—
heat2d	BF	BF	No OpenCL tgt impl.	No OpenCL src impl.	No OpenCL tgt impl.	No OpenCL src impl.	BF	—
iso2dfd	BF	BF	No OpenCL tgt impl.	No OpenCL src impl.	No OpenCL tgt impl.	No OpenCL src impl.	BF	—
jacobi	No OpenMP tgt impl.	No OpenMP src impl.	No OpenCL tgt impl.	No OpenCL src impl.	No src or tgt impl.	No src or tgt impl.	BF	—
HeCBench								
md	No OpenMP tgt impl.	No OpenMP src impl.	No OpenCL tgt impl.	No OpenCL src impl.	No src or tgt impl.	No src or tgt impl.	BF	—
nqueen	No OpenMP tgt impl.	No OpenMP src impl.	No OpenCL tgt impl.	No OpenCL src impl.	No src or tgt impl.	No src or tgt impl.	BF	—
page-rank	No OpenMP tgt impl.	No OpenMP src impl.	No OpenCL tgt impl.	No OpenCL src impl.	No src or tgt impl.	No src or tgt impl.	BF	—
scan	BF	BF	No OpenCL tgt impl.	No OpenCL src impl.	No OpenCL tgt impl.	No OpenCL src impl.	—	—
stencil1d	BF	—	No OpenCL tgt impl.	No OpenCL src impl.	No OpenCL tgt impl.	No OpenCL src impl.	—	—

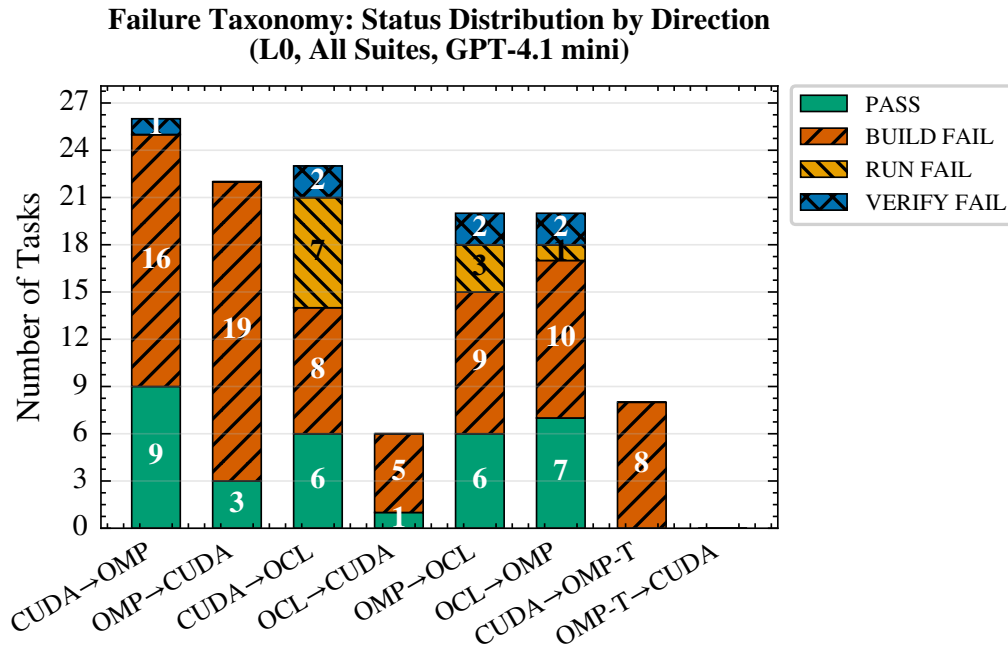


Figure 21: Failure taxonomy distribution (GPT-4.1 mini). Compare with Figure 2 in main body (Qwen 3.5 397B).

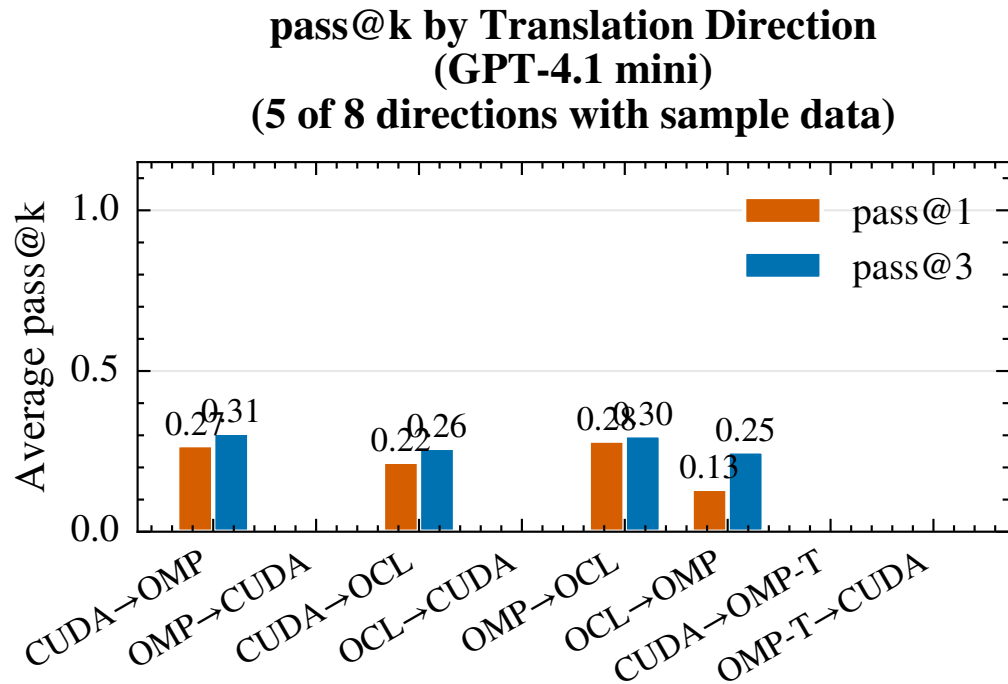


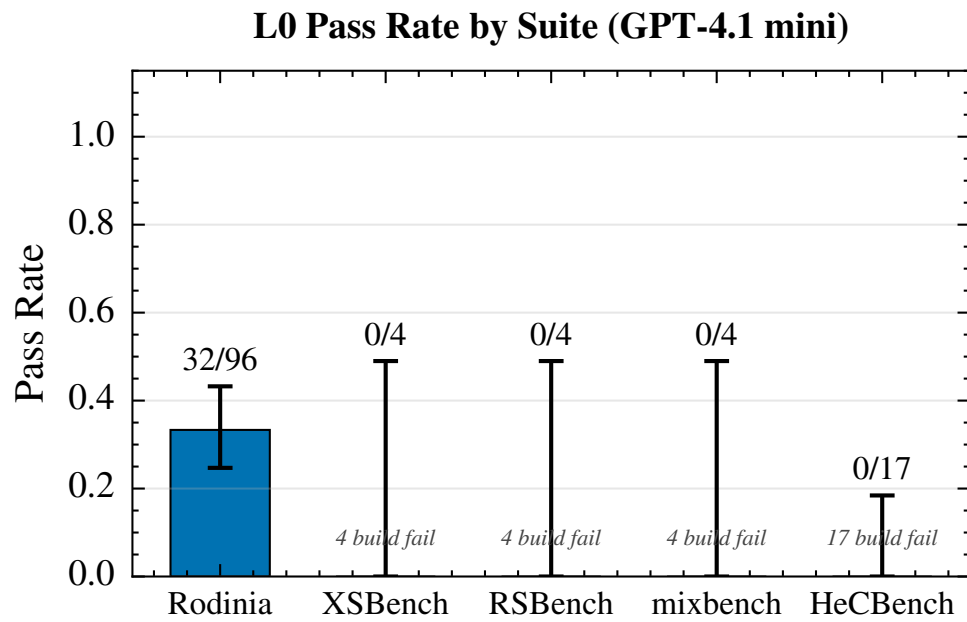
Figure 22: Pass@ k rates by translation direction (GPT-4.1 mini).

Table 17: Full per-kernel greedy pass rates across all 31 evaluation kernels (Qwen 3.5 397B-A17B, 710 primary campaign tasks, 31 kernels across 5 suites). KNOWN_FAIL source specs excluded.

Suite	Kernel	Total	PASS	Fail	Rate	95% Wilson CI
HeCBench	stencil1d	15	15	0	100.0%	[79.6%, 100.0%]
HeCBench	floydwarshall	20	16	4	80.0%	[58.4%, 91.9%]
HeCBench	heat2d	20	15	5	75.0%	[53.1%, 88.8%]
HeCBench	iso2dfd	20	15	5	75.0%	[53.1%, 88.8%]
Rodinia	hotspot	30	21	9	70.0%	[52.1%, 83.3%]
Rodinia	nn	10	7	3	70.0%	[39.7%, 89.2%]
Rodinia	hotspot3d	30	19	11	63.3%	[45.5%, 78.1%]
HeCBench	jacobi	10	6	4	60.0%	[31.3%, 83.2%]
HeCBench	md	10	6	4	60.0%	[31.3%, 83.2%]
HeCBench	nqueen	10	6	4	60.0%	[31.3%, 83.2%]
HeCBench	page-rank	10	6	4	60.0%	[31.3%, 83.2%]
Rodinia	particlefilter	30	17	13	56.7%	[39.2%, 72.6%]
Rodinia	bfs	30	16	14	53.3%	[36.1%, 69.8%]
Rodinia	cfid	30	16	14	53.3%	[36.1%, 69.8%]
Rodinia	lud	30	15	15	50.0%	[33.2%, 66.8%]
Rodinia	nw	30	15	15	50.0%	[33.2%, 66.8%]
Rodinia	pathfinder	30	15	15	50.0%	[33.2%, 66.8%]
Rodinia	backprop	30	13	17	43.3%	[27.4%, 60.8%]
Rodinia	srad	30	12	18	40.0%	[24.6%, 57.7%]
HeCBench	scan	15	5	10	33.3%	[15.2%, 58.3%]
mixbench	mixbench	30	8	22	26.7%	[14.2%, 44.4%]
Rodinia	bptree	30	4	26	13.3%	[5.3%, 29.7%]
Rodinia	lavamd	30	2	28	6.7%	[1.8%, 21.3%]
Rodinia	streamcluster	30	2	28	6.7%	[1.8%, 21.3%]
HeCBench	convolution1d	10	0	10	0.0%	[0.0%, 27.8%]
Rodinia	dwt2d	10	0	10	0.0%	[0.0%, 27.8%]
Rodinia	gaussian	10	0	10	0.0%	[0.0%, 27.8%]
Rodinia	heartwall	30	0	30	0.0%	[0.0%, 11.4%]
Rodinia	myocyte	30	0	30	0.0%	[0.0%, 11.4%]
RSBench	rsbench	30	0	30	0.0%	[0.0%, 11.4%]
XSBench	xsbench	30	0	30	0.0%	[0.0%, 11.4%]

Table 18: Qwen 3.5 evaluation campaign cost summary (March 27–April 2, 2026).

Metric	Value
Total API cost	\$145.37
Cost per task	\$0.13
Total tokens	120.6M (96.2M in, 24.3M out)
Campaign duration	7 days (1,136 tasks, 4,600 requests)



XSBench/RSBench/mixbench/HeCBench 0%: complex multi-file programs with inter-module dependencies — all translations failed (primarily build/linker errors).

Figure 23: Cross-suite pass rate comparison (GPT-4.1 mini).